Lightweight and Fast Time-Series Anomaly Detection via Point-Level and Sequence-Level Reconstruction Discrepancy

Lei Chen, *Member, IEEE*, Jiajun Tang, Ying Zou, Xuxin Liu, Xingquan Xie, and Guangyang Deng

***Abstract*—Unsupervised time-series anomaly detection (TSAD) aims to identify anomalies in industrial sensing signals to ensure production safety. As Industry 4.0 emerges, TSAD deployment must migrate from resource-rich cloud to resource-limited edges for real-time and fine-grained control. In this case, it raises new targets with high accuracy, high timeliness, and low consumption for TSAD. However, existing models focus solely on achieving high accuracy by building neural networks with deep structures and large parameters. Consequently, these models demand prohibitive training durations and computational overhead, which makes them unsuitable for edge deployment. To solve this issue, an unsupervised lightweight and fast TSAD model, namely, LFTSAD, is proposed via point-level and sequence-level reconstruction discrepancy in this article. *First*, to achieve high timeliness and low consumption, LFTSAD uses two two-layer multilayer perceptron networks (MLPs) to construct a lightweight contrastive architecture with few parameters. *Second*, leveraging the lightweight architecture, a dual-branch reconstruction network is designed to generate corresponding reconstruction discrepancies from point-level and sequence-level perspectives. *Finally*, a novel anomaly scoring scheme is designed to combine point-level and sequence-level reconstruction discrepancies for more accurate anomaly detection. To the best of our knowledge, this is the first work to develop a lightweight All-MLP-based TSAD model for resource-limited edge devices. Extensive experiments demonstrate that LFTSAD is 3–10 times faster in timeliness, consumes only 1/2 of the resources, and achieves accuracy that is either comparable to or superior to several deep SOTA models. The source code of LFTSAD is here https://github.com/infogroup502/LFTSAD**

***Index Terms*—All-multilayer perceptron network (MLP)-based anomaly detection, lightweight network and design, point-level and sequence-level, reconstruction discrepancy, time-series anomaly detection (TSAD).**

I. Introduction

TIME-series anomaly detection (TSAD) aims to identify outliers or deviations in temporal data that record the state of a continuous dynamic system [1]. Coming to Industry 4.0, time-series data exhibit new characteristics: complex temporal dynamics, rare anomaly labels, and rapidly increasing data volumes [2]. These characteristics undermine the effectiveness of traditional statistical, machine-learning-based, supervised, and semi-supervised TSAD models [3], compromising their competitiveness. As a result, unsupervised deep-learning-based TSAD models, such as those using CNN, transformer, and RNN architectures, have attracted significant attention in recent years [4], [5], [6], [7], [8]. However, the existing unsupervised TSAD models still face two main challenges.

A. Challenge

1) *Balancing Timeliness, Resource Efficiency, and Accuracy:* Traditional deep TSAD models are typically deployed on resource-rich cloud servers, where they prioritize high accuracy using deep architectures and large parameters [9], [10], [11], [12], [13], [14]. Consequently, they require long training and detection time, along with significant computational and storage resources. As Industry 4.0 emerges, industrial production processes require real-time and fine-grained intelligent monitoring and management. This makes it necessary to migrate the TSAD models from cloud servers to edge devices. However, edge devices typically have limited computational and storage resources, which restricts their ability to run complex deep models. For example, a Raspberry Pi 4b has only 2-GB RAM and 1.5-GHz clock speed. Therefore, unsupervised TSAD models need to find a tradeoff among timeliness, resource consumption, and accuracy to ensure stable operation on resource-constrained edge devices.
2) *Constructing Powerful Anomaly Scoring Scheme*: An unsupervised TSAD model typically requires an anomaly scoring scheme to evaluate observations at each timestamp and detect anomalies. The existing schemes can be mainly divided into three categories: reconstruction error-based [12], prediction error-based [9], and hybrid error-based [10]. However, each of these schemes has limitations, which makes it difficult to achieve both high accuracy and low time overhead, particularly when handling time-series data with complex features. As a result, these schemes struggle to perform effectively on resource-constrained edge devices.

B. Existing Solution

To address the first challenge, *some approaches* use parallel computing [15] or federated learning [16] to accelerate the speed of the existing deep TSAD models. However, they

Received 23 August 2024; revised 29 March 2025 and 17 April 2025; accepted 27 April 2025. Date of publication 19 May 2025; date of current version 4 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62103143 and Grant 62203164, in part by Hunan Provincial Natural Science Foundation of China under Grant 2024JJ5162, and in part by the Scientific Research Fund of Hunan Provincial Education Department under Grant 22B0471. *(Corresponding author: Lei Chen.)*

The authors are with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan 411201, China (e-mail: chenlei@hnust.edu.cn; jiajuntang@mail.hnust.edu.cn; 1040134@hnust.edu.cn; xuxinliu@mail.hnust.edu.cn; xingquanxie@mail.hnust.edu.cn; guangyangdeng@mail.hnust.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3565807

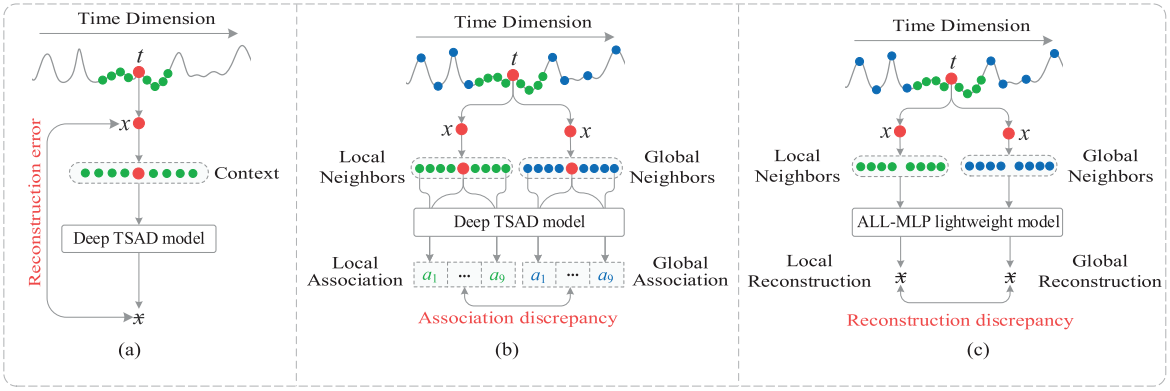

Fig. 1. Illustration of three anomaly scoring schemes. (a) Reconstruction error-based anomaly scoring. (b) Association discrepancy-based anomaly scoring. (c) Reconstruction discrepancy-based anomaly scoring.

require significant computational resources and large storage due to multithreaded processing and distributed learning. *Other methods* either apply compression techniques (pruning and quantization) to reduce redundant parameters [17], [18] or use shallow CNN or multilayer perceptron network (MLP) networks to replace time- or resource-consuming modules [19], [20] in the existing deep models. In this case, model parameters and computation times are reduced, achieving a balance between high accuracy and high timeliness. However, these methods still rely on deep structures, which limit their optimization capabilities in terms of time and resource efficiency. Thus, they are unsuitable for stable operation in resource-constrained environments.

To address the second challenge, a novel association discrepancy-based anomaly scoring scheme is introduced in 2022 [21], as shown in Fig. 1(b). In the proposed scheme, it states that compared with normal timestamps, anomalous timestamps in the time-series data exhibit stronger associations with local neighbors and weaker associations with global neighbors. Based on this work, several advanced TSAD models, such as AnomalyTrans (ATran) [21] and DCdetector (DCDet) [22], are proposed in the past two years, achieving high accuracy by capturing fine-grained short- and long-term contextual information. However, the computational complexity of fine-grained context processing leads to high time and resource consumption. Therefore, this novel scoring scheme is not well-suited for high-timeliness TSAD applications on resource-limited edge devices.

In summary, current acceleration schemes and accuracy optimization methods fail to achieve an effective tradeoff among speed, resource efficiency, and accuracy on resource-limited edge devices. To meet the demands of real-time TSAD at the edge devices, a more direct and effective solution is to design lightweight TSAD models using simple networks and shallow structures. Among neural networks, MLPs are particularly well-suited due to their simplicity and efficiency. This makes All-MLP architectures a promising choice for balancing timeliness, low resource consumption, and accuracy. Although some existing TSAD models, such as PatchAD (PaAD) [19] and USAD [20], adopt All-MLP designs, they still rely on deep structures. For example, PaAD uses a deep transformer with MLP units, while USAD builds an autoencoder using two multilayer MLPs. Therefore, a gap remains in developing TSAD models based solely on shallow All-MLP architectures (e.g., one or two layers) for edge deployment. However, shallow structures inherently offer limited representation capacity, making it challenging to simultaneously ensure high accuracy, fast inference, and low resource usage.

C. New Insight

To fill such a gap, this article aims to develop an unsupervised lightweight TSAD model based only on the two-layer MLP architecture for deployment on resource-limited edge devices. For this goal, two bottlenecks need to be addressed.

The first challenge lies in how to solely use two-layer MLPs to design a lightweight architecture with high speed, accuracy, and low consumption. To address this, the association discrepancy-based scoring scheme is used to guide architecture construction. However, traditional fine-grained association calculations are time- and resource-intensive, as each timestamp must evaluate associations with all the neighbors individually, as shown in Fig. 1(b). This complexity makes fine-grained association processing infeasible for a two-layer MLP network. To overcome this, a reconstruction-discrepancy-based scheme is proposed by fusing reconstruction-based and association discrepancy-based scoring schemes, as shown in Fig. 1(c). Instead of computing each association value explicitly, we focus on learning whether a strong association exists between a timestamp and its neighbors, significantly simplifying the process. Based on this approach, two two-layer MLPs are used to perform contrastive reconstruction learning of "local neighbors-to-a timestamp" and "global neighbors-to-same timestamp" respectively. A small difference between the two reconstruction outputs indicates strong associations with both local and global neighbors, suggesting a normal timestamp. In contrast, a large discrepancy is interpreted as an anomaly. To the best of our knowledge, the fusion of reconstruction error-based and association discrepancy-based scoring schemes has not been previously explored.

The second challenge is enhancing detection accuracy of the designed lightweight architecture. Typically, outliers in time series persist across multiple timestamps and can be categorized as explicit or implicit, as presented in Fig. 2. In explicit outliers [Fig. 2(a)], the association pattern of each anomalous timestamp (x_1) with its point-level neighbors deviates significantly from that of normal timestamps (y_1). In this case, the lightweight architecture can accurately identify anomalous timestamps. However, implicit outliers present a greater challenge, as only some timestamps exhibit association patterns that significantly deviate from normal timestamps, while others appear similar. As illustrated in Fig. 2(b), the association pattern of x_2 with its point-level neighbors closely

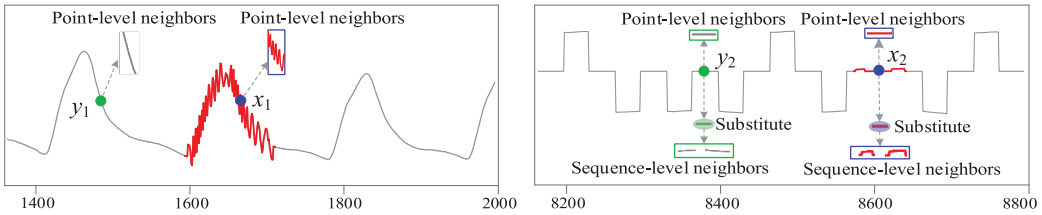

Fig. 2. Two types of anomalies in time series. (a) Explicit outlier. (b) Implicit outlier.

resembles that of y_2, increasing the risk of misclassification. To address this issue, we expand the analysis beyond individual timestamps. Instead of relying solely on x_2, we consider a subsequence centered around x_2 as its substitute, as shown in the elliptical region of Fig. 2(b). This extended view reveals that the association pattern of the substitute subsequence of x_2 with its sequence-level neighbors differs significantly from that of y_2. By incorporating both the point-level and sequence-level perspectives, our lightweight architecture enhances detection accuracy for both the explicit and implicit outliers.

Based on these insights, an unsupervised lightweight and fast TSAD model, namely, LFTSAD, is proposed via point-level and sequence-level reconstruction discrepancy. LFTSAD comprises four two-layer MLPs: two form the point-level reconstruction branch, and the other two constitute the sequence-level reconstruction branch. Reconstruction discrepancies from both the branches are combined to detect timestamp-level anomalies, enabling deployment on resource-limited edge devices.

D. Contribution

The main contributions of this work are outlined as follows.

1) A lightweight All-MLP-based TSAD model LFTSAD is proposed to identify anomalies quickly and efficiently on resource-limited edge devices. Specifically, LFTSAD uses four parallelizable two-layer MLPs to construct a lightweight dual-branch contrastive reconstruction network from both the point-level and sequence-level perspectives. To the best of our knowledge, this is the first work to develop a lightweight All-MLP-based TSAD model for resource-limited edge devices.
2) A novel reconstruction discrepancy-based anomaly scoring scheme is designed, which effectively combines point-level and sequence-level reconstruction discrepancies to accurately identify both the explicit and implicit anomalies.
3) Extensive experiments are conducted on 14 public datasets and two edge devices: Raspberry Pi 4b and NVIDIA Jetson Xavier NX. The experimental results demonstrate that LFTSAD outperforms several deep SOTA models in timeliness and resource consumption while achieving comparable or superior accuracy. Specifically, LFTSAD ranks first or second on more than half of the eight evaluation metrics across the 14 datasets. Moreover, its detection time is 3–10 times faster than that of deep SOTA models.

II. Related Work

Since our model is built upon the association discrepancy-based scoring scheme and All-MLP architecture, related work on these two topics is summarized as follows [5].

A. Association Discrepancy-Based TSAD

In 2022, association discrepancy-based anomaly scoring scheme was proposed for unsupervised timestamp-level anomaly detection [21]. In the past two years, several advanced TSAD models based on association discrepancy have been proposed and demonstrated strong performance [23], [24], [25].

For example, the ATran model [21] uses a learnable Gaussian kernel for local associations and a multihead attention mechanism for global associations. The MAN-QSM model [25] adds a mask learning mechanism into the association discrepancy learning process to enhance detection accuracy. The DCDet model [22] designs an attention-based dual-branch contrastive learning architecture to capture both the local and global associations. The Dual-TF model [23] further learns time-frequency granularity discrepancies to identify anomalies. The SiET model [24] considers the spatial association of multiple variables and uses spatiotemporal association discrepancies for anomaly detection.

In summary, the association discrepancy-based scoring scheme effectively identifies timestamp-level anomalies in time-series data. However, the fine-grained association calculation is time-consuming and resource-intensive. As a result, the existing association discrepancy-based TSAD models require extensive training and testing times, as well as substantial CPU/GPU and storage resources. Furthermore, these models are typically designed for resource-rich cloud environments. Consequently, they incur higher resource consumption and time overhead, making them unsuitable for stable deployment in resource-constrained edge environments.

B. All-MLP-Based Time-Series Analysis

In recent years, several deep TSAD models have used simple MLP networks to enhance model timeliness by replacing time-consuming or resource-intensive modules. For instance, the PaAD model [19] uses MLPs to replace the attention module, thereby reducing the time overhead of anomaly detection. Similarly, the ANNet model [26] optimizes the LSTM cell using the simple MLP. In addition, the USAD model [20] uses two multilayer MLPs to design an autoencoder network for anomaly detection. Overall, the existing All-MLP-based TSAD models are still in the early stages of research. Despite offering certain improvements in timeliness compared with other models, their enhancements remain insufficient. Moreover, these models still rely on deep architectures, resulting in high resource consumption. This constraint hinders their stable deployment on resource-limited edge devices.

In contrast, All-MLP-based models have rapidly emerged in time-series forecasting over the past two years. For example, the FiLM model [27] conducts extensive experiments

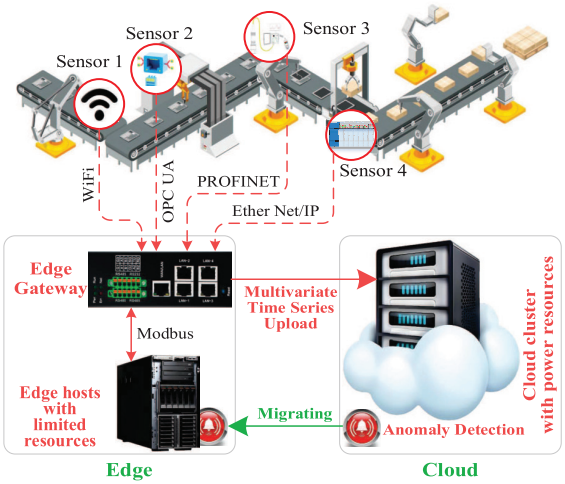

Fig. 3. Industry 4.0 scenario.

to demonstrate that MLP networks can replace the CNN or attention networks. Based on this finding, the Solar-Mixer model [28] constructs an All-MLP architecture to perform time-series forecasting. To enhance prediction performance, the HDMixer model [29] designs a stacked-MLP network to learn both the temporal and spatial features among multiple variables. However, these models still rely on deep structures and large parameters. Their timeliness and resource efficiency are inadequate, making them less competitive for deployment on resource-constrained edge devices. To address this issue, several customized lightweight All-MLP-based forecasting models have been proposed. For instance, the FreTS model [30] uses a shallow complex-valued MLP network to directly learn time-series features in the frequency domain. The FITS model [31] directly uses a two-layer MLP to simulate interpolation process in the frequency domain. Notably, the FITS model has fewer than 10 K parameters, achieving high speed, low consumption, and high accuracy.

In summary, forecasting models based on shallow All-MLP architecture demonstrate their effectiveness in timeliness, accuracy, and resource consumption. This highlights the feasibility of adopting similar architectures to design TSAD models that can operate stably on resource-limited edge devices. However, the existing TSAD models still rely on deep structures and large parameters, which prevent their direct deployment on resource-limited edge devices. Thus, a gap remains in developing a TSAD model that exclusively leverages a shallow All-MLP architecture for efficient and stable edge-based anomaly detection.

III. Preliminary

A. Industry 4.0 Scenario

In Industry 4.0, multiple heterogeneous sensors, such as vibration and angle sensors, are installed at different locations to enable intelligent and fine-grained monitoring and management of the production process, as shown in Fig. 3. Typically, these sensors collect different signals of industrial production at a fixed frequency. The acquired data, encoding industrial process dynamics, are transmitted via multiple network modalities (WiFi, 5G, Ethernet) and protocols (Modbus, UDP, TCP) to resource-constrained edge gateways or hosts. At the edge, sampled data are aggregated and forwarded to resource-rich cloud platforms for in-depth analysis. Advanced machine learning algorithms, particularly for anomaly detection, are used to extract insights from the collected data. However, the rapid expansion of cloud tasks in Industry 4.0 brings latency and resource-utilization challenges. To address these issues, many analysis models are being migrated from the cloud to the edge for high timeliness performance.

This article assumes that M sensors are used to gather the operating states of an industrial production process. After T collections, the generated data are a time series, formulated as $\mathbf{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T\} \in \mathbb{R}^{M \times T}$. Specifically, a time series consists of a time dimension and a variable dimension. For the time dimension, $\mathbf{X} = \{\boldsymbol{x}_t \in \mathbb{R}^{M \times 1}\}_{t=1}^{T}$, where x_t represents observations from M sensors at the tth timestamp. For the variable dimension, $\mathbf{X} = \{\boldsymbol{x}^m \in \mathbb{R}^{1 \times T}\}_{m=1}^{M}$, where x^m represents observations from T timestamps at the mth sensor.

B. Problem Statement

This article aims to build an LFTSAD model. First, a time series $\mathbf{X}$ is divided into an unlabeled training set $\mathbf{X}^1$ and a labeled test set $\mathbf{X}^2$. Second, the unlabeled $\mathbf{X}^1$ is used to train our lightweight model. Third, the trained model scores each timestamp in $\mathbf{X}^2$ as follows:

$$\text{Score}_t = \text{TSAD}(\boldsymbol{x}_t) \tag{1}$$

where $\boldsymbol{x}_t \in \mathbb{R}^{M \times 1}$ denotes the observations at the tth timestamp in $\mathbf{X}^2$. Finally, this score is used to determine whether a timestamp is an anomaly, as defined below

$$\bar{y}_t = \begin{cases} 1, & \text{if } \text{Score}_t \geq \lambda \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

where λ is the preset threshold, ranging from 0 to 1. If $\bar{y}_t = 1$, the tth timestamp is an anomaly; otherwise, it is normal. We aim for the predicted labels of each timestamp in $\mathbf{X}^2$ by our lightweight TSAD model to closely match the true labels.

IV. Proposed Model

A. Model Overview

To pursue triple goals of high accuracy, high timeliness, and low consumption, an unsupervised LFTSAD model is proposed. The overview of LFTSAD is presented in Fig. 4, which comprises three components. Note that the first two components complement each other and can be executed in parallel.

1) *Point-Level Reconstruction Discrepancy Learning:* As shown in Fig. 4(a), this component generates point-level reconstruction discrepancy for each timestamp from a point-level perspective. Technically, a customized sampling strategy is first designed to generate point-level local and global neighbors for each timestamp. Second, a shared two-layer MLP is used to perform "local neighbors-to-one timestamp" reconstruction learning. Third, another two-layer MLP is used to perform "global neighbors-to-same timestamp" reconstruction learning. Finally, the difference of two reconstruction values is point-level reconstruction discrepancy for a given timestamp.
2) *Sequence-Level Reconstruction Discrepancy Learning:* Complementing the first component, this one generates sequence-level reconstruction discrepancy for each timestamp from a sequence-level perspective, as

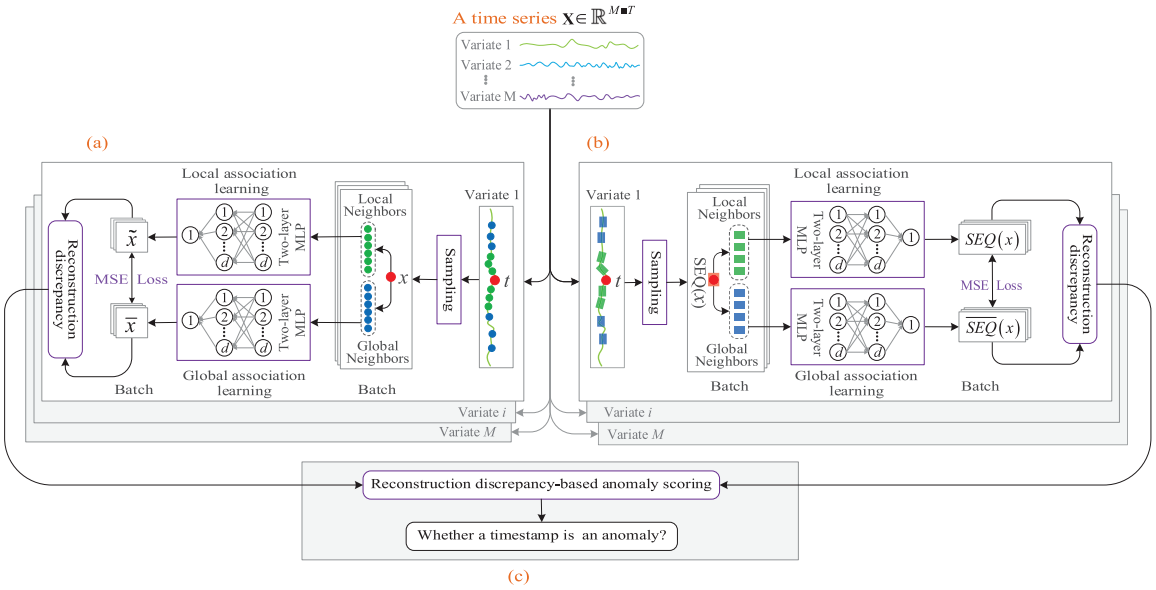

Fig. 4. Overview of the LFTSAD model. (a) Point-level reconstruction discrepancy learning. (b) Sequence-level reconstruction discrepancy learning. (c) Reconstruction discrepancy-based anomaly scoring.

shown in Fig. 4(b). Specifically, a customized sequence-level sampling strategy generates sequence-level local and global neighbors for each timestamp. Then, two two-layer MLPs are used to perform contrastive reconstruction learning on "local neighbors-to-a substitute sequence" and "global neighbors-to-same substitute sequence," respectively. Finally, the difference between two reconstruction sequences yields the sequence-level reconstruction discrepancy for a given timestamp.

3) *Reconstruction Discrepancy-Based Anomaly Scoring:* This one combines point-level and sequence-level reconstruction discrepancies to assess whether a timestamp is anomalous.

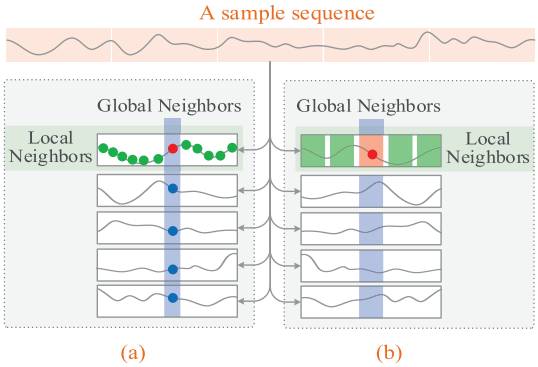

Fig. 5. (a) Point-level and (b) sequence-level sampling.

B. Point-Level Reconstruction Discrepancy Learning

As shown in Fig. 2, time-series data inherently exhibit various characteristics, including seasonal and periodic patterns, while outliers generally span multiple timestamps and significantly deviate from these normal patterns. In this context, a normal timestamp maintains associations not only with its adjacent timestamps (local neighbors) but also with distant timestamps (global neighbors), reflecting periodic and seasonal dependencies. In contrast, an anomalous timestamp is primarily associated with its local neighbors within the same outlier and lacks associations with distant global neighbors. Based on this principle, two two-layer MLPs are used to separately learn local and global associations for each timestamp from a point-level perspective, as presented in Fig. 4(a). Notably, the point-level learning process is variable-independent for multivariate time series. In this process, each variable is processed in parallel while sharing the same two MLPs.

1) Point-Level Sampling: A customized point-level sampling strategy is designed to quickly generate local and global neighbors for each timestamp, as shown in Fig. 5(a). The process involves four steps: 1) multiple overlapping sequences are randomly extracted using a look-back window of length WIN; 2) each sequence is divided into P_1 nonoverlapping segments, each segment containing P_2 timestamps, ensuring $P_1 \times P_2 = \text{WIN}$; 3) these segments are stacked into a 2-D matrix of size $P_1 \times P_2$; and 4) local neighbors of a timestamp at position (i, j) correspond to other timestamps in row i, while global neighbors are those in column j.

In the 2-D matrix, timestamps in the same row share local neighbors, while those in the same column share global neighbors. For multivariate time series, the sampling of each variable operates independently and can be executed in parallel, further enhancing efficiency.

2) Local Association Learning: Association calculation is typically a fine-grained process in which the relationship of each timestamp with its neighbors must be assessed. For example, if a timestamp has P_2 neighbors, P_2 associations must be computed. Such complexity exceeds the capability of a two-layer MLP. To address this issue, this article directly uses all the neighbors to reconstruct each timestamp. Accurate reconstruction indicates a strong association with its neighbors, while poor reconstruction suggests a weak association. This approach significantly simplifies the association calculation process while maintaining effectiveness.

Specifically, a two-layer MLP is used to learn local associations at each timestamp. This shallow MLP consists of an input layer with $P_2 - 1$ neurons, a hidden layer with d neurons, and an output layer with a single neuron. The input and hidden layers form the first fully connected layer (FCL), containing

$(P_2-1)\times d$ trainable parameters. The hidden and output layers constitute the second FCL, with $d\times 1$ trainable parameters. To speed up training, dropout and ReLU activation functions are applied to each neuron in the hidden layer.

For the tth timestamp of the mth variable, its local neighbors are defined as $LP(x_t^m)=\{x_i^m\}_{i=t-P_2}^{t-1}\in\mathbb{R}^{1\times(P_2-1)}$. Its local association learning process is detailed below.

The P_2-1 local neighbors are first fed into the first-FCL to learn local association features

$$\boldsymbol{h}_1\left(x_t^m\right)=\mathrm{ReLU}\left(LP\left(x_t^m\right)\times\mathbf{W}_1^1\right) \tag{3}$$

where $\boldsymbol{h}_1\in\mathbb{R}^{1\times d}$ is the output, and $\mathbf{W}_1^1\in\mathbb{R}^{(P_2-1)\times d}$ are the parameters of the first-FCL.

Then, $\boldsymbol{h}_1$ is input into the second-FCL to reconstruct x_t^m

$$\tilde{x}_t^m=\boldsymbol{h}_1\left(x_t^m\right)\times\mathbf{W}_2^1 \tag{4}$$

where $\tilde{x}_t^m\in\mathbb{R}$ is the local reconstruction value, and $\mathbf{W}_2^1\in\mathbb{R}^{d\times 1}$ are the parameters of the second-FCL.

3) Global Association Learning: Another two-layer MLP is adopted to capture global associations at each timestamp. This MLP has the same structure as the first MLP, comprising only two FCLs and $P_1\times d$ parameters. For the tth timestamp of the mth variable, its global neighbors are defined as $\mathrm{GP}(x_t^m)=\{x_{t-j\times P_2}^m\}_{j=1}^{P_1-1}\in\mathbb{R}^{1\times(P_1-1)}$. The global association calculation process is detailed as follows.

The P_1-1 global neighbors are input to the first-FCL to learn global associated features

$$\boldsymbol{h}_2\left(x_t^m\right)=\mathrm{ReLU}\left(\mathrm{GP}\left(x_t^m\right)\times\mathbf{W}_1^2\right) \tag{5}$$

where $\boldsymbol{h}_2\in\mathbb{R}^{1\times d}$ is the output, and $\mathbf{W}_1^2\in\mathbb{R}^{(P_1-1)\times d}$ are the parameters of the first-FCL.

Then, $\boldsymbol{h}_2$ is input into the second-FCL to reconstruct x_t^m

$$\bar{x}_t^m=\boldsymbol{h}_2\left(x_t^m\right)\times\mathbf{W}_2^2 \tag{6}$$

where $\bar{x}_t^m\in\mathbb{R}$ is the global reconstruction value, and $\mathbf{W}_2^2\in\mathbb{R}^{d\times 1}$ are the parameters of the second-FCL.

4) Loss: The local and global association learning processes form two complementary branches. To align their reconstruction values, the mean squared error (mse) loss is used as the contrastive learning objective

$$\mathrm{Loss}_P=\frac{1}{M}\sum_{m=1}^{M}\frac{1}{\mathrm{WIN}}\sum_{t=1}^{\mathrm{WIN}}\left(\tilde{x}_t^m-\bar{x}_t^m\right)^2 \tag{7}$$

where M is the number of variables, and WIN is the number of timestamps in one sample.

Notably, for each timestamp, the difference between the two reconstruction outputs defines the point-level reconstruction discrepancy.

C. Sequence-Level Reconstruction Discrepancy Learning

Outliers in time series can be categorized as explicit or implicit, as presented in Fig. 2. Compared with explicit outliers, implicit outliers exhibit more complex patterns. In implicit outliers, some timestamps share similar association patterns with normal data, while others exhibit significant deviations. In this context, point-level reconstruction discrepancy may misidentify anomalous timestamps. To address this issue, an additional reconstruction discrepancy learning process is introduced from a sequence-level perspective to enhance detection accuracy. Specifically, two two-layer MLPs are used to perform contrastive reconstruction on "local neighbors-to-a substitute sequence" and "global neighbors-to-same substitute sequence," respectively. The difference between the two reconstructed sequences yields the sequence-level reconstruction discrepancy for each timestamp.

1) Sequence-Level Sampling: A customized sequence-level sampling process is designed to quickly generate sequence-level local and global neighbors for each timestamp, as shown in Fig. 5(b). The process also consists of four steps: 1) multiple overlapping sequences of length WIN are randomly extracted from the time series $\mathbf{X}$ in parallel; 2) each sequence is divided into S_1 nonoverlapping segments, and each segment is further partitioned into S_2 nonoverlapping subsegments, each containing S_3 timestamps, ensuring $S_1\times S_2\times S_3=\mathrm{WIN}$; 3) the S_1 segments are stacked to form a 2-D matrix of size $S_1\times(S_2\times S_3)$; and 4) for a timestamp at row i and column j, its subsegment serves as its substitute. The remaining subsegments in row i are treated as local neighbors, while those in column j provide global neighbors.

2) Local Association Learning: A two-layer MLP network is used to perform "local neighbors-to-a substitute sequence" reconstruction from the sequence perspective. The input layer contains $(S_2-1)\times S_3$ neurons, the hidden layer has d neurons, and the output layer contains S_3 neurons. This MLP consists of two FCLs with a total of $S_2\times S_3\times d$ parameters. For the tth timestamp of the mth variable, the value is denoted as $x_t^m\in\mathbb{R}$. Its substitute sequence is $\mathrm{SEQ}(x_t^m)=\{x_i^m\}_{i=t-S_3}^{t}$, and local neighbors are $\mathrm{LS}(x_t^m)=\{x_i^m\}_{i=t-[(S_2-1)\times S_3]}^{t-S_3}$. The sequence-level local reconstruction process is described below.

First, the $(S_2-1)\times S_3$ local neighbors are input into the first-FCL to learn local association features

$$\boldsymbol{h}_3\left(x_t^m\right)=\mathrm{ReLU}\left(\mathrm{LS}\left(x_t^m\right)\times\mathbf{W}_1^3\right) \tag{8}$$

where $\boldsymbol{h}_3\in\mathbb{R}^{1\times d}$ is the output, and $\mathbf{W}_1^3\in\mathbb{R}^{[(S_2-1)\times S_3]\times d}$ are the parameters of the first-FCL.

Then, $\boldsymbol{h}_3$ is input into the second-FCL to reconstruct the substitute sequence $\mathrm{SEQ}(x_t^m)$ of the tth timestamp

$$\widetilde{\mathrm{SEQ}}\left(x_t^m\right)=\boldsymbol{h}_3\left(x_t^m\right)\times\mathbf{W}_2^3 \tag{9}$$

where $\widetilde{\mathrm{SEQ}}(x_t^m)\in\mathbb{R}^{1\times S_3}$ is the local reconstruction sequence, and $\mathbf{W}_2^3\in\mathbb{R}^{d\times S_3}$ are the parameters of the second-FCL.

3) Global Association Learning: Another two-layer MLP network is used to perform "global neighbors-to-same substitute sequence" reconstruction. The input layer contains $(S_1-1)\times S_3$ neurons, the hidden layer has d neurons, and the output layer contains S_3 neurons. This MLP has only $S_1\times S_3\times d$ parameters. For the tth timestamp of the mth variable, its global neighbors are defined as $GS(x_t^m)=\{\{x_i^m\}_{i=t-j\times(S_2\times S_3)-S_3}^{t-j\times(S_2\times S_3)}\}_{j=1}^{S_1-1}$.

First, the $(S_1-1)\times S_3$ global neighbors are input to the first-FCL to learn the global association features

$$\boldsymbol{h}_4\left(x_t^m\right)=\mathrm{ReLU}\left(GS\left(x_t^m\right)\times\mathbf{W}_1^4\right) \tag{10}$$

where $\boldsymbol{h}_4\in\mathbb{R}^{1\times d}$ is the output, and $\mathbf{W}_1^4\in\mathbb{R}^{[(S_1-1)\times S_3]\times d}$ are the parameters of first-FCL.

Next, $\boldsymbol{h}_4$ is further input into the second-FCL to reconstruct the substitute sequence of the tth timestamp

$$\overline{\mathrm{SEQ}}\left(x_t^m\right)=\boldsymbol{h}_4\left(x_t^m\right)\times\mathbf{W}_2^4 \tag{11}$$

where $\overline{\mathrm{SEQ}}(x_t^m)\in\mathbb{R}^{1\times S_3}$ is the global reconstruction sequence, and $\mathbf{W}_2^4\in\mathbb{R}^{d\times S_3}$ are the parameters of second-FCL.

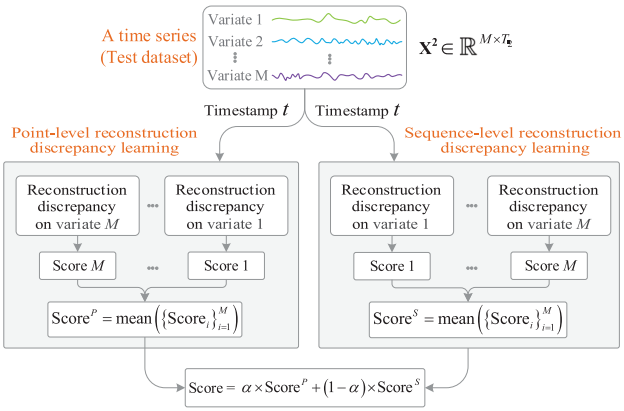

Fig. 6. Reconstruction discrepancy-based anomaly scoring.

4) Loss: The mse function is used again as the contrasting learning loss

$$\text{Loss}_S = \frac{1}{M}\sum_{m=1}^{M}\frac{1}{\text{WIN}}\sum_{t=1}^{\text{WIN}}\left(\widetilde{\text{SEQ}}\left(x_t^m\right) - \overline{\text{SEQ}}\left(x_t^m\right)\right)^2. \quad (12)$$

D. Reconstruction Discrepancy-Based Anomaly Scoring

As illustrated in Fig. 6, a novel reconstruction discrepancy-based anomaly scoring scheme is designed.

First, a to-be-tested timestamp $\boldsymbol{x}_t \in \mathbb{R}^{M\times 1}$ is input into the point-level reconstruction discrepancy learning component, where M variables are processed in parallel to produce M point-level reconstruction discrepancies. The final point-level discrepancy for $\boldsymbol{x}_t$ is calculated as the average of these values, formulated as $\text{Score}^P(\boldsymbol{x}_t) = \text{mean}(\{\text{Score}_m^P\}_{m=1}^M)$, where Score_m^P denotes the point-level discrepancy on the mth variable. *Second*, the to-be-tested timestamp $\boldsymbol{x}_t$ is fed into the sequence-level learning component to produce M sequence-level reconstruction discrepancies. The final sequence-level discrepancy is produced as the average of these values, formulated as $\text{Score}^S(\boldsymbol{x}_t) = \text{mean}(\{\text{Score}_m^S\}_{m=1}^M)$. *Third*, the final anomaly score for $\boldsymbol{x}_t$ is generated as follows:

$$\text{Score}(\boldsymbol{x}_t) = \alpha \times \text{Score}^P(\boldsymbol{x}_t) + (1-\alpha) \times \text{Score}^S(\boldsymbol{x}_t) \quad (13)$$

where α is a predefined hyperparameter within $[0 \sim 1]$. Setting $\alpha = 0$ considers only the sequence-level discrepancy, while $\alpha = 1$ considers only the point-level discrepancy. *Finally*, the anomaly score is used in (2) to determine whether timestamp $\boldsymbol{x}_t$ is anomalous.

E. Algorithm and Complexity Analysis

The pseudocode for LFTSAD is shown in Algorithm 1.

Complexity Analysis: The computational cost of LFTSAD comprises three components: point-level reconstruction, sequence-level reconstruction, and anomaly scoring.

1) *Point-Level Reconstruction Learning:* This component consists of three stages: point-level sampling, local reconstruction, and global reconstruction. In the sampling stage, local and global neighbors are generated for each timestamp across all the M variables in parallel. The computational complexity of this process is $\mathcal{O}(T \times (P_1 + P_2))$, where T is the number of timestamps, and P_1 and P_2 represent the numbers of global and local neighbors per timestamp, respectively. Next, two independent two-layer MLPs are used for local and global reconstruction, respectively. The first MLP reconstructs each timestamp using its local neighbors, while the second MLP uses global neighbors for reconstruction. Both the operations are performed in parallel across the M variables to improve computational efficiency. For local reconstruction via the first MLP, the computational complexity is

$$\mathcal{O}\left(T \times \left(\underbrace{(P_2-1)\times d}_{\text{1st-layer}} + \underbrace{1 \times d}_{\text{2nd-layer}}\right)\right)$$

where (P_2-1), d, and 1 represent the number of neurons in the input, hidden, and output layers, respectively. The first and second layers contribute $(P_2-1)\times d$ and $1 \times d$ operations. Similarly, for global reconstruction using the second MLP, the complexity is

$$\mathcal{O}\left(T \times \left(\underbrace{(P_1-1)\times d}_{\text{1st-layer}} + \underbrace{1 \times d}_{\text{2nd-layer}}\right)\right)$$

with $(P_1-1)\times d$ and $1\times d$ corresponding to the operations in the first and second layers, respectively. In summary,

Algorithm 1 LFTSAD

input : Time-series data $\mathbf{X} \in \mathbb{R}^{M\times T}$, Training set $\mathbf{X}^1$, Test set $\mathbf{X}^2$, and $\mathbf{X} = \mathbf{X}^1 \cup \mathbf{X}^2$; parameter α.
output : Anomaly labels $\bar{y}$ for all timestamps in $\mathbf{X}^2$.

for $t \in \{1, \ldots, T\}$ **in** $\mathbf{X}^1$ **do**
  // Variable-independent learning
  for $m \in \{1, \ldots, M\}$ **parallel do**
    // **Point-level reconstruction**
    // Sampling
    $LP(x_t^m) \leftarrow$ generate local neighbors for x_t^m;
    $GP(x_t^m) \leftarrow$ generate global neighbors for x_t^m;
    // Local and global reconstruction
    $\tilde{x}_t^m \leftarrow$ 1st-two-layer-MLP$(LP(x_t^m))$;
    $\bar{x}_t^m \leftarrow$ 2nd-two-layer-MLP$(GP(x_t^m))$;
    // Loss function
    $Loss_P \leftarrow \tilde{x}_t^m = \bar{x}_t^m$ by 7;
    // **Sequence-level reconstruction**
    // Sampling
    $SEQ(x_t^m) \leftarrow$ generate substitute sequence for x_t^m;
    $LS(x_t^m) \leftarrow$ generate local neighbors for x_t^m;
    $GS(x_t^m) \leftarrow$ generate global neighbors for x_t^m;
    // Local and global reconstruction
    $\widetilde{SEQ}(x_t^m) \leftarrow$ 3rd-two-layer-MLP$(LP(x_t^m))$;
    $\overline{SEQ}(x_t^m) \leftarrow$ 4th-two-layer-MLP$(GP(x_t^m))$;
    // Loss function
    $Loss_S \leftarrow \widetilde{SEQ}(x_t^m) = \overline{SEQ}(x_t^m)$ by 12;
  end
end
// **Anomaly scoring and detection**
for $t \in \{1, \ldots, T\}$ **in** $\mathbf{X}^2$ **do**
  $Score^P(x_t) = mean\left(\{Score_m^P = Loss_P(x_t^m)\}_{m=1}^M\right)$
  $Score^S(x_t) = mean\left(\{Score_m^S = Loss_S(x_t^m)\}_{m=1}^M\right)$
  $Score(x_t) = \alpha \times Score^P(x_t) + (1-\alpha) \times Score^S(x_t)$
  $\bar{y}_t$ = anomaly detection$(Score(x_t))$ by 2;
end
Return $\bar{y}$.;

the overall complexity of this component is $\mathcal{O}(T \times (P_1 + P_2) \times (d+1))$.

2) *Sequence-Level Reconstruction Learning:* This component also includes three stages: sequence-level sampling, local reconstruction, and global reconstruction. *First*, sequence-level sampling generates local and global neighbors for each timestamp. Its complexity is $\mathcal{O}(T \times (S_1 + S_2 + S_3))$, where S_1 and S_2 are the numbers of global and local neighbors, and S_3 is the length of the substitute sequence. *Second*, the third two-layer MLP performs local reconstruction using sampled local neighbors. The complexity is

$$\mathcal{O}\left(T \times \left(\underbrace{((S_2-1)\times S_3)\times d}_{\text{1st-layer}} + \underbrace{S_3 \times d}_{\text{2nd-layer}}\right)\right)$$

where $(S_2 - 1) \times S_3$, d, and S_3 represent the numbers of neurons in the input, hidden, and output layers, respectively. The first layer contributes $(S_2-1)\times S_3 \times d$ operations, and the second layer contributes $S_3 \times d$. *Finally*, the fourth two-layer MLP performs global reconstruction using sampled global neighbors. The complexity is

$$\mathcal{O}\left(T \times \left(\underbrace{((S_1-1)\times S_3)\times d}_{\text{1st-layer}} + \underbrace{S_3 \times d}_{\text{2nd-layer}}\right)\right).$$

In summary, the overall complexity of this component is $\mathcal{O}(T \times (S_1 + S_2) \times S_3 \times d + T \times (S_1 + S_2 + S_3))$.

3) *Anomaly Scoring:* Based on the outputs of point-level and sequence-level reconstruction, two reconstruction discrepancies are computed and merged to produce the final anomaly score for each timestamp. This score determines whether the timestamp is anomalous. The computational complexity is $\mathcal{O}(T \times 2M)$, where T is the number of timestamps and M is the number of variables Notably, the first two components are carried out in parallel, and $P_1 + P_2 < (S_1 + S_2) \times S_3$. *In summary*, by ignoring relatively small terms—$\mathcal{O}(T \times 2M)$, $\mathcal{O}(T \times (S_1+S_2+S_3))$, and $\mathcal{O}(T\times(P_1+P_2)\times(d+1))$—the overall computational complexity of the proposed LFTSAD model is approximated as: $\mathcal{O}(T \times (S_1 + S_2) \times S_3 \times d)$. This complexity is nearly linear with respect to T, since $(S_1 + S_2) \times S_3 \times d \ll T$ in practical scenarios.

V. Experiment

A. Experimental Goal

The following questions are addressed in the experiments.

1) *RQ-1 (Accuracy)*: Can the model outperform baselines on univariate and multivariate time-series anomaly detection?
2) *RQ-2 (Deployability and Timeliness)*: How efficient is the model in terms of timeliness and resource consumption on resource-constrained edge devices?
3) *RQ-3 (Ablation)*: Does each proposed component contribute to overall performance?
4) *RQ-4 (Sensitivity)*: How sensitive is the model to different parameter settings?

TABLE I
Datasets

	Dataset	Features	Train Set	Test Set	URL
Multivariate	MSL	55	58317	73729	MSL-URL
	GECCO	9	69260	69261	GECCO-URL
	SWAN	38	60000	60000	SWAN-URL
	PSM	25	132480	87840	PSM-URL
	SMAP	25	135183	427617	SMAP-URL
	SMD	38	708405	708420	SMD-URL
	SWAT	51	494999	449918	SWaT-URL
	WADI	127	103680	69121	WADI-URL
Unvariate	ECG	1	182319	45580	ECG-URL
	MITDB	1	519999	130000	MITDB-URL
	SVDB	1	184319	46080	SVDB-URL
	UCR	1	2300	5200	UCR-URL
	UCR-AUG	1	2300	5200	UCR-AUG-URL
	Occupancy	1	8143	2665	Occupancy-URL

B. Dataset

In all, 14 publicly available real-world time-series datasets are used, including six univariate and eight multivariate datasets, as listed in Table I. These datasets span multiple domains to ensure fair and unbiased evaluation.

1) Multivariate Datasets: *MSL* records the status of multiple sensors on the Mars rover. *GECCO* contains drinking water quality data from IoT sensors. *SWAN* provides temporal data from solar photospheric vector magnetograms. *PSM* includes 25-D monitoring data from eBay server machines. *SMAP* presents soil samples and telemetry data from Mars rovers. *SMD* stores resource usage traces from 28 servers over five weeks. *SWAT* collects 51-D sensor data from a public water treatment system. *WADI* offers 127-D monitoring data from an industrial control system.

2) Univariate datasets. *ECG* contains electrocardiogram data with ventricular premature beats. *MITDB* includes 48 half-hour two-channel ECG recordings. *SVDB* consists of 78 half-hour ECG tracings with supraventricular arrhythmia. *UCR* features natural sequences with single anomalies. *UCR-AUG* provides augmented data based on the original UCR datasets. *Occupancy* records the information of temperature, humidity, light, and CO_2 levels.

C. Baseline and Metric

1) Baseline: To validate the effectiveness of the LFTSAD, 16 representative state-of-the-art (SOTA) models are selected as baselines, as listed in Table II. These baselines are selected based on their network structure, scoring scheme, and use of All-MLP architecture, where GDN, MAD-GAN (MGAN), and CSTGL are customized for multivariate time series and cannot be applied to univariate time series.

1) *Network Structure*: DCDet, PaAD, ATran, DTAAD, USAD, TGFLOW (MTGF), CSTGL, GDN, MGAN, OmniAnomaly (Omni), and NASA-LSTM (LSTM) adopt deep structures with numerous parameters. In contrast, IForest, FADSD, ADNEv, LTFAD, COUTA, and LFTSAD (Ours) use shallow structures with few parameters.
2) *Scoring Scheme*: LFTSAD, ATran, DCDet, and PaAD use association discrepancy-based scoring schemes. The other models rely on reconstruction error-based or prediction error-based scoring schemes.

TABLE II
BASELINES

Models	Full Name	Year	Source Code
PaAD [19]	PatchAD: A Lightweight Patch-Based MLP-Mixer for Time Series Anomaly Detection	2025	github.com/EmorZz1G/PatchAD
FADSD [32]	Frequency-Domain Spectrum Discrepancy-Based Fast Anomaly Detection for IIoT Sensor Time-Series Signals	2025	github.com/infogroup502/FADSD
LTFAD [33]	A lightweight All-MLP time–frequency anomaly detection for IIoT time series	2025	github.com/infogroup502/LTFAD
DTAAD [34]	Dual Tcn-Attention Networks for Anomaly Detection in Multivariate Time Series Data	2024	github.com/Yu-Lingrui/DTAAD
ADNEv [35]	AD-NEv: A Scalable Multilevel Neuroevolution Framework for Multivariate Anomaly Detection	2024	github.com/neuroevolution-agh/AD-NEv
CSTGL [36]	Correlation-Aware Spatial–Temporal Graph Learning for Multivariate Time-Series Anomaly Detection	2024	github.com/huankoh/CST-GL
COUTA [37]	Calibrated One-class Classification for Unsupervised Time Series Anomaly Detection	2024	github.com/xuhongzuo/couta
DCdet [22]	DCdetector: Dual Attention Contrastive Representation Learning for Time Series Anomaly Detection	2023	github.com/DAMO-DI-ML/KDD2023-DCdetector
MTGF [38]	MTGFLOW:Detecting Multivariate Time Series Anomalies with Zero Known Label	2023	github.com/zqhang/MTGFLOW
ATran [21]	Anomaly transformer: Time series anomaly detection with association discrepancy	2022	github.com/thuml/Anomaly-Transformer
GDN [39]	Graph Neural Network-Based Anomaly Detection in Multivariate Time Series	2021	github.com/d-ailin/GDN
USAD [20]	USAD: Unsupervised Anomaly Detection on Multivariate Time Series	2020	github.com/OpsPAI/MTAD
MGAN [40]	MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks	2019	github.com/Guillem96/madgan-pytorch
Omni [41]	OmniAnomaly: Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network	2019	github.com/NetManAIOps/OmniAnomaly
LSTM [42]	NASA-LSTM: Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding	2018	github.com/khundman/telemanom
IForest [43]	Isolation Forest	2008	github.com/OpsPAI/MTAD

3) *All-MLP Architecture*: LFTSAD, PaAD, LTFAD, and USAD are based on All-MLP architectures. Among them, LFTSAD and LTFAD adopt shallow architectures, whereas PaAD and USAD use deeper ones.

2) Metric: For comprehensive evaluation, eight widely used metrics [2], [44], [45] are used: Accuracy (Acc), Precision (Pre), Recall (Rec), $F1$-pointwise ($F1$), Affiliation-$F1$ ($F1_{\text{Af}}$), $F1$-point-adjusted ($F1_{\text{PA}}$), VUS_ROC (V_ROC), and VUS_PR (V_PR). All the metrics range from 0 to 1, with higher values indicating better TSAD performance.

D. Experimental Environment

All the experiments are conducted on a Windows 11 system with an Intel Core i7-10700KF CPU, NVIDIA GeForce RTX 3090 GPU (24-GB GPU-RAM), and 64-GB RAM. The LFTSAD model is implemented in Python using PyCharm and is publicly available at here. Official Python implementations of baseline models are obtained from the authors' repositories, as listed in Table II. The grid-search-based autotuning method is used to find the optimal parameters for all the models.

E. RQ-1 (Accuracy)

To answer *RQ-1*, LFTSAD is evaluated against 16 baselines on 14 datasets, including eight multivariate and six univariate datasets, using eight evaluation metrics.

Tables III and IV present the detection accuracy of 17 models on 14 datasets. In two tables, bold text indicates the best performance, while underlined text denotes the second-best performance. From two tables, several findings can be observed.

1) *Overall Performance*: LFTSAD, DTAAD, ADNev, DCDet, FADSD, ATran, MGAN, LTFAD, and Omni outperform other baselines. For example, on the MSL dataset, LFTSAD achieves an Acc of 0.9907, Pre of 0.9509, and V_ROC of 0.9300, while USAD scores only 0.7299 for Acc, 0.7198 for Pre, and 0.6102 V_ROC.
2) *Stability*: LFTSAD, ADNev, ATran, LTFAD, DCdet, and DTAAD show the most stable performance across eight metrics, followed by GDN, PaAD, FADSD, Omni, and LSTM. For example, on the ECG and UCR datasets, LFTSAD ranks first three times and second once.
3) *Anomaly Scoring Scheme*: Association discrepancy-based models, such as LFTSAD, DCDet, AnomalyTrans, and PaAD, outperform others. For example, on the SWAT and WADI datasets, DCdet and ATran surpass DTAAD, CSTGL, and Omni.
4) *Deep or Shallow Structures*: Deep models generally outperform shallow ones (COUTA, FADSD, IForest). However, shallow models like LFTSAD, ADNev, and LTFAD outperform several deep models. For example, on the Occupancy dataset, $F1_{\text{PA}}$ scores are 0.9919 for LFTSAD, 0.9877 for ADNEv, 0.8679 for PaAD, 0.9048 for DTAAD, and only 0.6811 for IForest.
5) *All-MLP Architecture:* LFTSAD and LTFAD are better than PaAD and USAD.
6) *In Summary:* LTFSAD achieves top-two rankings in more than one-half of the eight metrics, but its performance in traditional $F1$ scores is weaker. These results highlight the effectiveness of the lightweight All-MLP architecture and reconstruction discrepancy-based scoring.

F. RQ-2 (Deployability and Timeliness)

To address *RQ-2*, a PC and two resource-limited edge devices are selected to deploy the 17 models and detect anomalies on the MSL dataset. In edge devices, the Raspberry Pi 4b has a 1.5-GHz ARM Cortex-A72 processor and 2-GB RAM, while the Jetson Xavier NX is equipped with a six-core Carmel ARMv8.2 processor with 8 GB of RAM.

1) Timeliness and Resource Consumption: Table V lists the timeliness and resource consumption of the 17 models on PC and two edge devices, where *Ot* indicates out-of-memory. Several key observations are made:

TABLE III
ACCURACY ON MULTIVARIATE DATASETS

		Shallow models						Deep models										
		Ours	ADNEv	COUTA	FADSD	LTFAD	IForest	PaAD	DTAAD	CSTGL	DCdet	MTGF	ATran	GDN	USAD	MGAN	Omni	LSTM
MSL	*Acc*	**0.9907**	0.9294	0.9731	0.9856	0.9900	0.9001	0.9765	0.9814	0.9431	0.9905	0.9542	0.9859	0.9371	0.7299	0.9511	0.9863	0.9744
	Pre	0.9509	**0.9882**	0.8338	0.9281	0.9243	0.5589	0.9205	0.8807	0.9474	0.9508	0.9155	0.9339	0.9484	0.7198	0.7828	0.9323	0.8602
	Rec	0.9942	0.9334	0.9687	0.9361	**0.9959**	0.2454	0.8502	0.9698	0.9860	0.9681	0.6251	0.9436	0.9357	0.7760	0.9525	0.9693	0.9740
	F1	0.4632	0.5961	0.5144	0.4779	0.4807	0.4748	0.5029	0.5205	**0.6345**	0.4767	0.5038	0.4796	0.4730	0.5641	0.4883	0.4729	0.5070
	$F1_{PA}$	**0.9721**	0.8993	0.9465	0.9321	0.9588	0.4314	0.8840	0.9366	0.9311	0.9567	0.7430	0.9328	0.5550	0.4688	0.8594	0.9455	0.9136
	$F1_{Af}$	0.6631	0.5983	0.6774	0.6533	0.6961	0.3766	0.6874	0.3461	0.6631	0.6651	0.5992	0.6777	0.5160	0.6130	0.6975	**0.8620**	0.7148
	V_ROC	**0.9300**	0.8302	0.8242	0.8701	0.9075	0.5257	0.8914	0.5426	0.8888	0.8901	0.6520	0.8644	0.5121	0.6102	0.9186	0.9196	0.9040
	V_PR	**0.9236**	0.8841	0.7462	0.8558	0.8876	0.3800	0.7860	0.5177	0.8987	0.9155	0.6713	0.8936	0.5703	0.4760	0.8976	0.8029	0.8642
GECCO	*Acc*	**0.9921**	0.9804	0.9918	0.9920	0.9910	0.9887	0.9917	0.9830	0.9152	0.9844	0.9862	0.9779	0.9913	0.9207	0.9449	0.8451	0.9687
	Pre	0.7153	0.3192	0.9179	0.9453	0.6195	0.4590	0.9590	**0.9625**	0.9311	0.3817	0.3925	0.2456	0.6675	0.9293	0.2415	0.5124	0.5414
	Rec	0.4233	0.7616	0.3521	**0.9982**	0.3836	0.3836	0.7552	0.3521	0.8577	0.5955	0.3668	0.5973	0.3521	0.5494	0.7147	0.6454	0.2473
	F1	0.5027	0.5882	**0.7177**	0.4736	0.5034	0.6228	0.682	0.5608	0.6855	0.5019	0.6599	0.4965	0.5308	0.6299	0.4973	0.5104	0.5675
	$F1_{PA}$	**0.5318**	0.4498	0.5089	0.4710	0.4738	0.4179	0.4805	0.5155	0.5217	0.4561	0.3792	0.3841	0.4610	0.4906	0.3610	0.4209	0.3395
	$F1_{Af}$	0.5103	0.7406	0.3356	0.5314	0.5594	0.3083	0.2314	0.2365	0.6612	0.6532	0.3073	0.6668	**0.9292**	0.4795	0.6610	0.6686	0.3443
	V_ROC	0.5395	0.6554	0.5236	**0.9822**	0.5405	0.5240	0.5158	0.5192	0.4832	0.6054	0.5252	0.5675	0.6251	0.5887	0.7151	0.5069	0.5234
	V_PR	0.4138	0.3455	0.5168	**0.9608**	0.3691	0.2998	0.5087	0.5274	0.505	0.3007	0.2526	0.1864	0.6077	0.6252	0.3816	0.5187	0.3222
SWAN	*Acc*	**0.8647**	0.8463	0.6764	0.8642	0.8625	0.8441	0.8552	0.8640	0.8412	0.8609	0.7639	0.8641	0.6740	0.8592	0.8041	0.7550	0.8400
	Pre	**1**	0.8771	0.5022	0.9807	0.9733	0.7422	0.9986	0.9950	0.8433	0.9680	0.9933	0.9786	0.5020	0.9897	0.3634	0.5755	0.8400
	Rec	0.5850	0.6146	0.8191	0.5939	0.5944	0.7995	0.5867	0.5859	0.8010	0.5915	0.4512	0.5987	0.5235	0.8592	0.5528	0.9469	**1**
	F1	0.4365	**0.7503**	0.6092	0.4153	0.4109	0.3695	0.4433	0.4873	0.5125	0.4462	0.3854	0.4159	0.4027	0.5238	0.4755	0.4561	0.4565
	$F1_{PA}$	0.7832	0.7227	0.6227	0.7398	0.7381	0.7698	0.7392	0.7375	**0.8362**	0.7343	0.6205	0.7429	0.3405	0.7243	0.5798	0.7159	0.7113
	$F1_{Af}$	0.1611	0.2216	0.6263	0.1547	0.0702	0.661	0.0155	0.1897	0.6874	0.0393	0.0154	0.0883	0.1768	0.7124	0.1772	0.8076	**0.9002**
	V_ROC	0.8376	0.8377	0.8827	0.8608	0.8614	0.8779	0.8389	0.8462	0.7437	0.8248	0.8542	0.8622	0.8088	0.558	0.7918	**0.8901**	0.5437
	V_PR	0.9129	0.9265	0.9202	0.9356	0.9362	0.9209	0.9296	0.9310	0.8920	0.9033	**0.9465**	0.9370	0.9114	0.8582	0.7864	0.9252	0.9222
PSM	*Acc*	**0.9884**	0.9282	0.7543	0.9864	0.9881	0.8318	0.9509	0.9351	0.9351	0.9881	0.9430	0.9803	0.8273	0.4690	0.9654	0.9423	0.8650
	Pre	0.9649	0.9443	0.5306	0.9747	0.9712	0.6299	0.9865	0.9852	0.9311	0.9784	**1**	0.9710	0.9970	0.9428	0.9261	0.9063	0.7353
	Rec	0.9922	0.7878	**0.9948**	0.9763	0.9864	0.9555	0.9199	0.7780	0.9800	0.9741	0.8380	0.9840	0.3788	0.4358	0.9816	0.8836	0.9356
	F1	0.4390	**0.9276**	0.6675	0.4264	0.4294	0.5418	0.4265	0.4444	0.2518	0.4327	0.4235	0.4280	0.4197	0.3418	0.4394	0.4749	0.6102
	$F1_{PA}$	0.9784	0.8590	0.6920	0.9755	0.9781	0.7593	0.9567	0.8694	0.9274	0.9762	0.9119	**0.9825**	0.5490	0.5960	0.9530	0.8948	0.8234
	$F1_{Af}$	0.7064	0.6692	0.7732	0.6411	0.6835	0.6599	0.5789	0.2890	0.7207	0.5357	0.4516	0.6551	0.4360	0.7706	0.5467	0.7262	**0.7982**
	V_ROC	**0.9149**	0.8869	0.7902	0.8223	0.8885	0.7324	0.8056	0.7619	0.6162	0.7030	0.7000	0.8986	0.5763	0.5166	0.9141	0.8508	0.7928
	V_PR	**0.9279**	0.7923	0.7508	0.8667	0.9123	0.7137	0.8725	0.9183	0.7126	0.8796	0.8180	0.9206	0.7046	0.7749	0.9242	0.8949	0.7956
SMAP	*Acc*	**0.9900**	0.9336	0.8526	0.9847	0.9896	0.7549	0.9429	0.9811	0.9784	0.9819	0.9340	0.9826	0.9467	0.2746	0.9169	0.9394	0.9689
	Pre	**0.9590**	0.9255	0.4645	0.9389	0.9389	0.3411	0.9565	0.8229	0.8107	0.9511	0.8876	0.9545	0.9522	0.2208	0.6636	0.9541	0.9303
	Rec	0.9635	0.5233	**1**	0.9514	0.9828	0.8833	0.9084	**1**	0.9278	0.9419	0.5548	0.9892	0.6005	**1**	0.9549	0.5532	0.8465
	F1	0.4692	**0.7533**	0.5497	0.4658	0.4732	0.5002	0.4669	0.4662	0.3142	0.4668	0.4698	0.4693	0.4671	0.2594	0.4944	0.4681	0.4655
	$F1_{PA}$	**0.9612**	0.6685	0.6344	0.9355	0.9604	0.5065	0.9504	0.9028	0.8619	0.9564	0.6828	0.9516	0.7424	0.3618	0.7830	0.7003	0.8864
	$F1_{Af}$	0.6778	0.5102	0.6751	0.6762	0.6516	0.6054	0.5054	0.5237	0.6555	0.5337	0.5046	0.6149	**0.7166**	0.7053	0.6376	0.6556	0.5231
	V_ROC	0.8478	0.6427	**0.8996**	0.8028	0.8458	0.7706	0.5821	0.6389	0.8069	0.6196	0.6358	0.8422	0.6864	0.5471	0.8424	0.6138	0.7728
	V_PR	**0.8508**	0.7566	0.7219	0.8332	0.8484	0.5963	0.6305	0.7478	0.7774	0.6487	0.6363	0.8416	0.7160	0.6167	0.8501	0.6447	0.7770
SMD	*Acc*	**0.9900**	0.9604	0.9807	0.9819	0.9870	0.9480	0.9826	0.9837	0.9430	0.9858	0.9749	0.9809	0.9504	0.4959	0.9169	0.9816	0.8563
	Pre	0.8692	0.8496	0.7277	0.7876	0.8127	0.3790	0.7648	0.8396	0.6141	0.8534	0.7543	**0.8791**	0.5491	0.1481	0.6636	0.7555	0.3826
	Rec	0.8997	0.3628	0.8562	0.7719	0.8933	0.3934	0.8383	**1**	0.8400	0.8995	0.5971	0.9587	0.3552	0.7644	0.9549	0.8239	**1**
	F1	0.5054	**0.6893**	0.6042	0.4940	0.4956	0.5456	0.5816	0.5733	0.5801	0.4926	0.5232	0.5026	0.5157	0.4091	0.4944	0.5077	0.5486
	$F1_{PA}$	0.8842	0.3556	0.7867	0.7797	0.8511	0.3860	0.7999	0.8778	0.7202	0.8688	0.6666	**0.8918**	0.2721	0.2482	0.7830	0.7882	0.5535
	$F1_{Af}$	0.6907	0.4613	**0.7723**	0.6614	0.6465	0.4006	0.7451	0.6688	0.6615	0.6392	0.5712	0.7222	0.3611	0.6374	0.6376	0.7569	0.7623
	V_ROC	0.7221	0.5129	0.7245	0.6208	0.7184	0.5298	0.6993	0.6408	0.5109	0.6461	0.5892	0.8098	0.5129	0.5416	0.8524	0.6904	**0.8701**
	V_PR	0.6813	0.6314	0.6192	0.5445	0.6496	0.2761	0.6098	0.4789	0.5768	0.6089	0.5099	0.6488	0.3314	0.4205	**0.7510**	0.5970	0.6573
SWAT	*Acc*	**0.9929**	0.9749	0.9608	0.9880	0.9912	0.6491	0.9759	0.9598	0.8644	0.9911	0.9583	0.9832	0.9245	0.7299	0.9169	0.8565	0.9584
	Pre	0.9452	0.9828	0.9974	0.9417	0.9322	0.5818	0.9723	**1**	0.9011	0.9668	0.9992	0.8782	0.9900	0.5749	0.6636	0.4511	0.9985
	Rec	**1**	0.8072	0.6790	0.9602	**1**	0.6909	0.8285	0.6879	0.6894	0.9932	0.6568	**1**	0.8572	0.9952	0.9549	0.8387	0.6798
	F1	0.4696	**0.8762**	0.8551	0.4719	0.4727	0.4715	0.6244	0.4403	0.7250	0.4763	0.5461	0.4793	0.8261	0.8441	0.4944	0.7499	0.7987
	$F1_{PA}$	**0.9718**	0.8864	0.8080	0.9509	0.9649	0.7350	0.8946	08151	0.7867	0.9708	0.7926	0.9352	0.8462	0.9477	0.7830	0.5866	0.8089
	$F1_{Af}$	0.6637	0.6949	0.1996	0.5814	0.6858	0.4920	0.6132	0.5636	0.3587	0.6878	0.0466	0.7135	0.6499	0.1037	0.6376	**0.7343**	0.2393
	V_ROC	**0.9835**	0.6883	0.5313	0.9117	0.9825	0.5062	0.5949	0.5874	0.5001	0.9782	0.5132	0.9782	0.6830	0.5206	0.8524	0.6653	0.5526
	V_PR	0.9619	0.7208	0.5890	0.8976	0.9555	0.4465	0.4157	0.4052	0.5574	**0.9665**	0.5752	0.9289	0.7280	0.5825	0.7510	0.4962	0.6114
WADI	*Acc*	**0.9956**	0.9708	0.9707	0.9872	0.993	0.9674	0.9732	0.9523	0.8797	0.9947	0.9777	0.9842	0.9665	0.8922	0.9169	0.6590	0.9605
	Pre	0.9234	0.9277	0.6326	0.8366	0.8737	0.7896	0.7217	0.5193	0.7371	0.9116	0.9841	0.7537	**0.9942**	0.3075	0.6636	0.4920	0.6060
	Rec	**1**	**1**	0.9393	0.9129	**1**	0.4434	0.7241	0.6888	0.6241	**1**	0.5505	**1**	0.6792	0.7001	0.9549	**1**	0.9252
	F1	0.4901	0.6346	0.7477	0.4922	0.4939	0.2710	0.4570	0.4946	**0.8566**	0.4915	0.6535	0.5028	0.3100	0.3394	0.4944	0.1650	0.6956
	$F1_{PA}$	**0.9602**	0.5879	0.756	0.8731	0.9326	0.5679	0.7229	0.8769	0.6292	0.9538	0.7061	0.8596	0.4726	0.4273	0.7830	0.7938	0.7323
	$F1_{Af}$	0.7300	0.5703	**0.7913**	0.6914	0.6997	0.6581	0.7119	0.5225	0.7881	0.7206	0.6436	0.7437	0.5917	0.6967	0.6376	0.7187	0.7238
	V_ROC	0.9566	0.5580	0.8654	0.8177	0.9555	0.5785	0.6858	0.7106	0.7888	**0.9584**	0.5979	0.9540	0.5430	0.6569	0.8524	0.5115	0.8589
	V_PR	**0.9235**	0.5475	0.7083	0.7507	0.9007	0.5180	0.5719	0.8065	0.8458	0.9200	0.6137	0.8426	0.5660	0.3906	0.7510	0.5290	0.6945

TABLE IV
ACCURACY ON UNIVARIATE DATASETS

		Shallow models						Deep models							
		Ours	ADNEv	COUTA	FADSD	LTFAD	IForest	PaAD	DTAAD	DCdet	MTGF	ATran	USAD	Omni	LSTM
ECG	*Acc*	**0.9897**	0.9652	0.7360	0.9802	0.9842	0.6738	0.9737	0.9866	0.9885	0.9777	0.9833	0.7039	0.9763	0.9798
	Pre	0.6481	0.5943	0.3754	0.6412	0.6385	0.3360	0.6257	**0.8603**	0.5530	0.6302	0.5859	0.4830	0.5642	0.6355
	Rec	**1**	**1**	**1**	0.5756	0.8108	0.6844	0.8480	0.9548	0.8750	0.5714	0.6471	0.5549	0.7985	0.9818
	F1	0.5085	**0.6245**	0.5364	0.4992	0.4991	0.1002	0.5150	0.5541	0.5003	0.5013	0.4949	0.1696	0.5033	0.4961
	$F1_{PA}$	0.7865	0.5699	0.5458	0.6240	0.7447	0.4012	0.7154	**0.8575**	0.6625	0.5994	0.6855	0.3346	0.6612	0.7821
	$F1_{Af}$	0.6672	0.6431	0.7510	0.6528	0.6653	0.4356	0.7155	0.6638	0.6345	0.6514	0.6343	0.6968	**0.7843**	0.6523
	V_ROC	**0.8420**	0.8417	0.8118	0.6628	0.7375	0.6984	0.7854	0.8417	0.6832	0.6435	0.5925	0.5064	0.7466	0.7564
	V_PR	0.6868	0.5116	0.7076	0.5271	0.6026	0.4936	0.7551	**0.7710**	0.4352	0.4905	0.4139	0.5491	0.7550	0.7507
MITDB	*Acc*	**0.9970**	0.9518	0.9685	0.9871	0.9934	0.9383	0.9810	0.9893	0.9952	0.9903	0.9932	0.9459	0.9901	0.9903
	Pre	**0.8922**	0.8349	0.4341	0.7099	0.7854	0.6572	0.5605	0.7736	0.8391	0.7811	0.7811	0.7035	0.7087	0.7401
	Rec	**1**	**1**	**1**	0.7933	**1**	0.4442	**1**	0.7891	**1**	**1**	**1**	0.7236	**1**	**1**
	F1	0.4978	0.5122	0.5103	0.4967	0.4972	0.4981	**0.5210**	0.5020	0.4994	0.5036	0.4975	0.1334	0.5063	0.5014
	$F1_{PA}$	**0.9430**	0.8493	0.8467	0.7493	0.8798	0.5196	0.8125	0.8337	0.8492	0.7813	0.8271	0.7432	0.7478	0.6054
	$F1_{Af}$	0.6662	**0.6851**	0.6613	0.6481	0.6706	0.6645	0.5733	0.6727	0.6821	0.6752	0.6684	0.6705	0.6298	0.6450
	V_ROC	**0.9474**	0.9267	0.9341	0.7734	0.9450	0.6412	0.8178	0.8736	0.9466	0.9063	0.9441	0.5015	0.9435	0.8991
	V_PR	**0.8975**	0.8212	0.7700	0.6390	0.8431	0.4008	0.7447	0.8700	0.8712	0.7738	0.8594	0.5202	0.8350	0.8261
SVDB	*Acc*	**0.9952**	0.9949	0.9890	0.9870	0.9903	0.9869	0.9895	0.9948	0.9861	0.9902	0.9891	0.9803	0.9942	0.9872
	Pre	0.7894	**1**	0.6204	0.6218	0.6505	0.5520	0.9382	0.9884	0.7060	0.6608	0.6503	0.3392	0.8270	0.6672
	Rec	**1**	0.7200	**1**	0.7115	**1**	0.9714	**1**	0.7200	**1**	0.8606	0.8582	**1**	0.8618	0.8580
	F1	0.5074	0.5492	0.5626	0.5010	0.5000	0.2012	0.5391	0.5411	**0.5977**	0.5001	0.4982	0.1667	0.5205	0.5073
	$F1_{PA}$	**0.8823**	0.8372	0.7658	0.6637	0.7883	0.4455	0.8748	0.8331	0.6218	0.7757	0.7399	0.7422	0.8440	0.7506
	$F1_{Af}$	0.7762	0.8418	0.8449	0.6814	0.7417	0.5577	0.8630	0.8399	0.6489	0.8348	0.7410	0.6746	**0.8988**	0.7458
	V_ROC	**0.9012**	0.6701	0.8971	0.7544	0.8974	0.5505	0.8228	0.6700	0.8703	0.8180	0.8404	0.5010	0.8252	0.8506
	V_PR	**0.8039**	0.7777	0.7170	0.5780	0.7316	0.4300	0.6621	0.6719	0.5637	0.6816	0.6752	0.5376	0.7452	0.6946
UCR	*Acc*	**0.9994**	0.9701	0.9751	0.9965	0.9949	0.9595	0.9942	0.9990	0.9962	0.9935	0.9958	0.5491	0.9190	0.9799
	Pre	**0.9714**	0.9463	0.7300	0.8500	0.7294	0.6630	0.7727	0.9356	0.8430	0.8166	0.8226	0.6355	0.3950	0.4184
	Rec	**1**	**1**	**1**	**1**	**1**	0.6697	**1**	**1**	**1**	**1**	**1**	0.8281	**1**	**1**
	F1	0.5404	**0.6417**	0.5069	0.5000	0.5022	0.1083	0.4927	0.5400	0.5090	0.5126	0.5126	0.1587	0.4909	0.4931
	$F1_{PA}$	**0.9855**	0.9206	0.6239	0.9389	0.8435	0.5261	0.8718	0.9373	0.9148	0.7926	0.9027	0.6133	0.6264	0.5900
	$F1_{Af}$	**0.8938**	0.7428	0.8726	0.8816	0.6639	0.5806	0.5757	0.7891	0.7023	0.6264	0.6778	0.6684	0.7329	0.6440
	V_ROC	0.8655	0.9535	0.7960	0.8618	**0.9662**	0.4996	0.7560	0.8994	0.8649	0.7716	0.8616	0.5005	0.9100	0.8241
	V_PR	**0.8557**	0.6273	0.8330	0.7291	0.8341	0.2018	0.6841	0.7476	0.7692	0.2838	0.8131	0.5322	0.7673	0.6800
UCR-AUG	*Acc*	**0.9988**	0.9375	0.9349	0.9377	0.9944	0.7935	0.8744	0.9287	0.9938	0.9663	0.9965	0.9438	0.8977	0.9736
	Pre	0.9976	0.8378	0.7924	0.9926	0.9433	0.6965	0.9966	0.5377	0.9919	0.8893	**1**	0.9852	0.8640	0.7910
	Rec	**1**	0.4117	**1**	0.8767	**1**	0.4117	0.7844	**1**	0.9953	0.5521	**1**	**1**	0.9306	**1**
	F1	0.3448	**0.8976**	0.7082	0.3519	0.4799	0.5118	0.3696	0.6213	0.3584	0.5577	0.3549	0.3080	0.4986	0.4968
	$F1_{PA}$	**0.9988**	0.8961	0.8211	0.9311	0.9708	0.5419	0.8779	0.6993	0.9936	0.6813	0.9964	0.9711	0.5521	0.8833
	$F1_{Af}$	0.8921	0.6598	0.8506	0.8646	0.6608	0.6894	0.5205	0.7301	0.8025	0.7449	0.7269	0.7715	**0.8942**	0.6721
	V_ROC	**0.9562**	0.9171	0.9420	0.8562	0.9539	0.5981	0.6900	0.8555	0.9304	0.6460	0.9124	0.5041	0.8811	0.9487
	V_PR	**0.9821**	0.9413	0.8981	0.9345	0.9341	0.5431	0.8551	0.9644	0.9675	0.8415	0.9592	0.7270	0.9248	0.8666
Occupancy	*Acc*	**0.9932**	0.9853	0.9637	0.7781	0.9716	0.8064	0.7865	0.9725	0.9856	0.8502	0.9906	0.8891	0.9795	0.9816
	Pre	**0.9901**	0.9614	0.9048	0.9631	0.9823	0.5669	0.8435	0.9314	0.9706	0.9817	0.9865	0.8891	0.9440	0.9589
	Rec	0.9938	0.9234	0.9392	0.3602	0.9377	0.8529	0.6958	0.8796	0.8626	0.9541	0.8933	**1**	0.7992	0.9891
	F1	0.3771	**0.8987**	0.7486	0.4070	0.4052	0.4232	0.2357	0.4002	0.1438	0.1830	0.4118	0.4706	0.4017	0.5004
	$F1_{PA}$	**0.9919**	0.9877	0.8494	0.5243	0.9595	0.6811	0.8679	0.9048	0.9135	0.9324	0.9421	0.9413	0.9656	0.9786
	$F1_{Af}$	0.6366	0.5619	0.7268	0.5961	0.6812	0.1162	0.5187	0.7009	0.3677	0.3542	0.5638	**0.9190**	0.5706	0.9010
	V_ROC	**0.9444**	0.8735	0.8943	0.6644	0.9086	0.5768	0.8014	0.8718	0.9238	0.8534	0.8788	0.7886	0.8613	0.9342
	V_PR	0.9681	0.9642	0.9244	0.7826	0.9413	0.7508	0.9065	0.9540	0.9355	0.9758	0.9278	0.9612	0.9119	**0.9822**

1) *Model Parameters*: IForest, FADSD, and LFTSAD have the fewest and second-fewest parameters. Overall, shallow models require significantly fewer parameters than deep models. For example, the number of parameters is 0 for IForest, 12.6 K for LFTSAD, 4741 K for ATran, and 258.6 K for CSTGL.
2) *Timeliness*: Shallow models (LFTSAD, ADNEv, LTFAD, FADSD, COUTA, IForest) show faster training and testing times compared with deep models. On the PC, with a batch size of 128, LFTSAD completes one epoch in 26.5 s while DTAAD and DCdet take 148 and 271.8 s respectively.
3) *Computing and Storage Consumption:* Shallow models consume significantly fewer CPU/GPU and RAM/GPU-RAM resources. For instance, on Jetson Xavier NX, CPU usage is 38% for LFTSAD, compared with 53.8% for CSTGL and 62.2% for GDN. On Raspberry Pi 4B (2 GB RAM), DCdet uses 402-MB RAM, while LFTSAD uses only 185 MB.

2) Deployability: Fig. 7 shows the deployment results on Raspberry Pi 4b, where all the models except PaAD are successfully deployed. Lightweight and shallow models are easier to deploy compared with deep models. Moreover, due to frequent production process changes, deployed models must be updated regularly. Lightweight models like IForest, FADSD, and LFTSAD can be updated directly on edge devices with minimal data and communication overhead.

3) Summary: Thanks to its lightweight All-MLP architecture with four parallel two-layer MLPs, LFTSAD runs efficiently and stably on resource-constrained edge devices.

TABLE V
TIMELINESS AND RESOURCE CONSUMPTION ANALYSIS

		Shallow models						Deep models									
		Ours	ADNEv	COUTA	FADSD	LTFAD	IForest	PaAD	DTAAD	CSTGL	DCdet	MTGF	ATran	GDN	USAD	MGAN	Omni	LSTM
PC	Model parameters	12.6*K*	95.3*K*	86.1*K*	0	84.2*K*	0	1477.9*K*	195.6*K*	258.6*K*	301.5*K*	132.6*K*	4741*K*	227.2*K*	213.6*K*	214.6*K*	189*K*	249*K*
	One-epoch-train-time	26.5*s*	43.2*s*	65.1*s*	0	42.7*s*	19.7*s*	905.9*s*	148.0 *s*	139.8*s*	271.8*s*	105.2*s*	201.1*s*	159.5*s*	233.5*s*	178.1*s*	127.9*s*	158.9*s*
	All-point-test-time	43.2*s*	68*s*	76.8*s*	35.6*s*	65.2*s*	22.8*s*	958.4*s*	226.4*s*	126.6*s*	425.8*s*	125.6*s*	325.9*s*	118.6*s*	143.9*s*	162.3*s*	147.6*s*	120.4*s*
	CPU Usage	35%	34.9%	37.3%	3.2%	37.6%	16.7%	38.5%	37.7%	45.2%	65.0%	47.0%	51.0%	45.0%	47.0%	52.6%	44.2%	58.7%
	GPU Usage	11%	32.6%	17.5%	21.0%	16.7%	0.0%	83.3%	74.2%	47.8%	59.0%	44.0%	62.0%	58.0%	46.0%	58.2%	48.6%	45.8%
	RAM Usage	8.4*GB*	11.2*GB*	10.6*GB*	2.5*GB*	8.1*GB*	7.4*GB*	16.3*GB*	9.5*GB*	15.4*GB*	18.1*GB*	11.9*GB*	18.4*GB*	10.7*GB*	9.8 *GB*	15.2*GB*	13.8*GB*	12.5*GB*
	GPU-RAM Usage	2.2*GB*	8.9*GB*	1.2*GB*	3.1*GB*	4.6*GB*	0.7*GB*	12.8*GB*	7.8*GB*	9.6*GB*	15.3*GB*	7.4*GB*	16.7*GB*	8.7*GB*	7.1*GB*	9.2*GB*	9.1*GB*	7.8*GB*
Jetson Xavier NX	10000-point-test-time	24.1*s*	43.5*s*	35.2*s*	18.6*s*	31.7*s*	8.5*s*	ot	95.6*s*	98.4*s*	230*s*	92*s*	314.6*s*	147.6*s*	94.8*s*	106.2*s*	115.6*s*	108.3*s*
	CPU Usage	38%	41.8%	39.30%	42.1%	43.1%	35%	ot	66%	53.8%	68.2%	52%	74.2%	62.2%	54.6%	54.8%	54.5%	51.7%
	RAM Usage	4.2*GB*	6.8*GB*	6.2*GB*	3.7*GB*	6.6*GB*	3.8*GB*	ot	6.2*GB*	6.6*GB*	7.1*GB*	5.8*GB*	7.6*GB*	7.0*GB*	6.5*GB*	6.8*GB*	6.6*GB*	6.5*GB*
Raspberry Pi 4b	1000-point-test-time	42.2*s*	84.4*s*	93.2*s*	34.8*s*	68.3*s*	32.2*s*	ot	328.6*s*	213.8*s*	425.1*s*	124.6*s*	458.6*s*	105.8*s*	112.0*s*	204.2*s*	223.3*s*	194.3*s*
	CPU Usage	36.8%	43.2%	37.3%	35.9%	42.5%	30.8%	ot	67.7%	62%	72.2%	51%	75.8%	53.9%	53%	60.7%	61.2%	54.2%
	RAM Usage	185*MB*	201*MB*	192*MB*	133*MB*	204*MB*	125*MB*	ot	318*MB*	275*MB*	402*MB*	236*MB*	461*MB*	241.9*MB*	223*MB*	266*MB*	268*MB*	228*MB*

PC - (CPU: i7-10700KF, GPU: RTX 3090, RAM: 64G, GPU-RAM: 24G); **Jetson Xavier NX** - (CPU: Carmel ARMv8.2, RAM: 8G);
Raspberry Pi 4b - (CPU: ARM Cortex-A72, RAM: 2G)

TABLE VI
EFFECTIVENESS OF THREE ABLATION EXPERIMENTS

		ECG			MITDB			MSL			PSM			SWAN		
		Pre	*Rec*	$F1_{PA}$	*Pre*	*Rec*	$F1_{PA}$	*Pre*	*Rec*	$F1_{PA}$	*Pre*	*Rec*	$F1_{PA}$	*Pre*	*Rec*	$F1_{PA}$
(a)	Only point-level learning	0.5681	0.8790	0.6992	0.8721	1	0.9310	0.9212	0.9443	0.9321	0.9631	0.9871	0.9650	0.9882	0.5846	0.7381
	Only sequence-level learning	0.5752	0.8592	0.6981	0.8330	1	0.9051	0.9506	0.9371	0.9480	0.9387	0.9668	0.9511	1	0.5638	0.7380
	Point-level and sequence-level joint learning	0.6481	1	0.7865	0.8922	1	0.9430	0.9509	0.9942	0.9721	0.9649	0.9922	0.9784	1	0.5850	0.7832
(b)	Mean-reconstruction-error-based scoring	0.5586	0.7501	0.6411	0.8000	0.9310	0.8889	0.8851	0.9420	0.9350	0.9402	0.9376	0.9338	0.9201	0.5140	0.7338
	Max-reconstruction-error-based scoring	0.5559	0.8550	0.6800	0.8339	0.9438	0.9100	0.9178	0.9207	0.9189	0.9378	0.9231	0.9358	0.9201	0.4954	0.7141
	Reconstruction discrepancy-based scoring	0.6481	1	0.7865	0.8922	1	0.9430	0.9509	0.9942	0.9721	0.9649	0.9922	0.9784	1	0.5850	0.7832
(c)	two-layer-MLP-based architecture	0.6481	1	0.7865	0.8922	1	0.9430	0.9509	0.9942	0.9721	0.9649	0.9922	0.9784	1	0.5850	0.7832
	two-layer-RNN-based architecture	0.5162	0.6313	0.5680	0.8084	0.5665	0.6662	0.9063	0.9676	0.9360	0.9599	0.9566	0.9582	0.9996	0.5271	0.6902
	two-layer-CNN-based architecture	0.5144	0.5581	0.5353	0.8798	0.8982	0.8889	0.9679	0.9100	0.9380	0.9533	0.9787	0.9659	0.9996	0.5295	0.6923

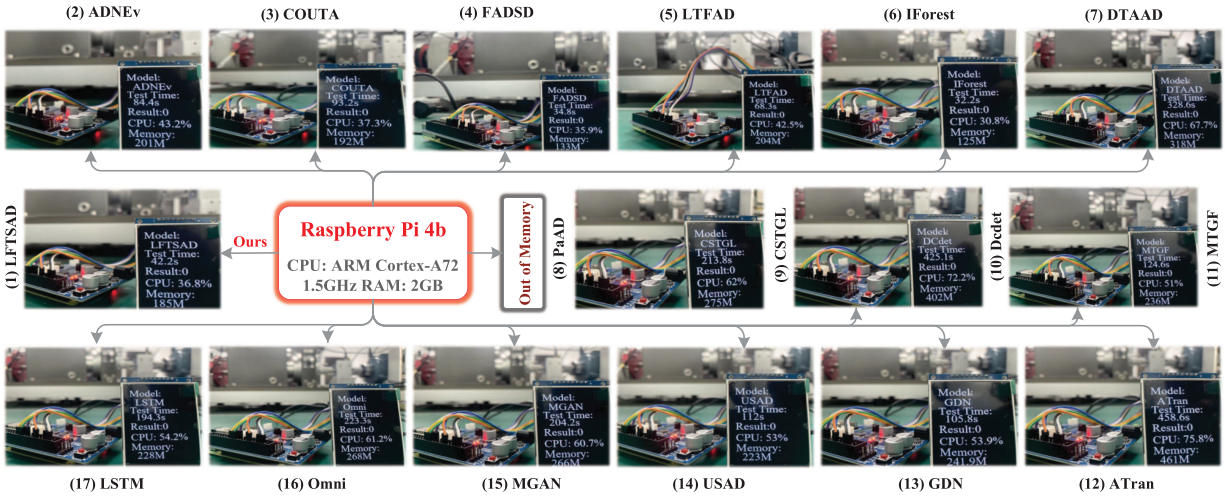

Fig. 7. Deployment results on Raspberry Pi 4b.

G. RQ-3 (Ablation)

To answer *RQ-3*, three ablation experiments are conducted. Specifically, two univariate datasets (ECG and MITDB), three multivariate datasets (MSL, PSM, and SWAN), and three metrics (Pre, Rec, and $F1_{\text{PA}}$) are chosen for this experiment.

1) Point-Level and Sequence-Level Learning: These two components are essential to LFTSAD, complementing each other to improve detection accuracy. In this experiment, three configurations are compared: 1) only point-level learning; 2) only sequence-level learning; and 3) point-level and sequence-level joint learning. Table. VI(a) presents the detection performance of the three configurations.

1) *Overall Performance:* Joint learning consistently outperforms individual configurations across all the metrics and datasets. For example, on the MSL dataset, $F1_{\text{PA}}$ is 0.9321 for point level, 0.9480 for sequence level, and 0.9721 for joint learning.
2) *Comparison of Individual Configurations:* Each configuration shows advantages on different datasets and metrics. For instance, point-level learning achieves better

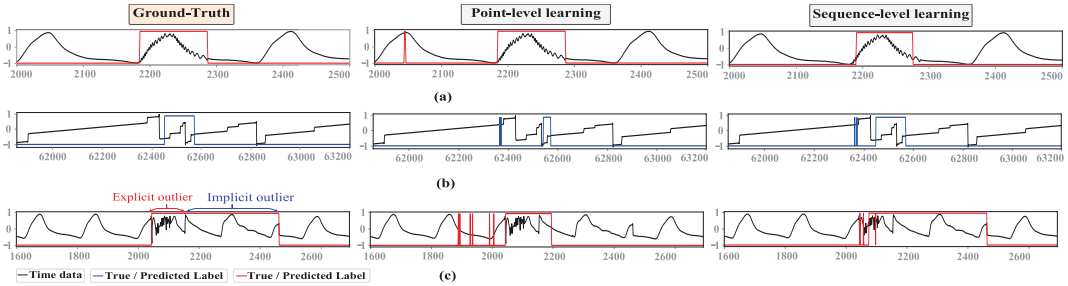

Fig. 8. Detection results for explicit and implicit outliers. (a) Explicit outlier in the univariate dataset UCR (2180–2280). (b) Implicit outlier in the first variable of MSL dataset MSL (62450–62560). (c) Mixed outlier in the univariate dataset UCR_AUG (1600–2600).

Pre and $F1_{PA}$ on MITDB, while sequence-level learning performs better on MSL. This implies that the two configurations complement each other.

To further evaluate these two components, an explicit outlier from UCR, an implicit outlier from the first variable of MSL, and a mixed outlier from UCR_AUG are extracted. Fig. 8 shows plots of the detection results of these two components on the three outliers. It is clear that point-level learning performs better in detecting explicit outliers, while sequence-level learning is more effective for implicit outliers. For example, on the explicit outlier in UCR, point-level learning accurately identifies the entire outlier but misclassifies a normal timestamp. In contrast, sequence-level learning partially detects the outlier while avoiding false positives. In summary, these results demonstrate that point-level and sequence-level joint learning enhances anomaly detection accuracy by leveraging their complementary strengths.

2) Reconstruction Discrepancy-Based Anomaly Scoring: This is another key component of LFTSAD. In this experiment, it is compared with two alternatives: the average reconstruction error-based scheme and the maximum reconstruction error-based scheme. Table. VI(b) illustrates the effectiveness of the three scoring schemes.

1) *Overall Performance:* The reconstruction discrepancy-based scheme outperforms both the alternatives. It achieves the highest scores across all the metrics and datasets, with an average improvement of 6.51%.
2) *Comparison of Alternatives*: The maximum error-based scheme yields better Pre on ECG, MITDB, and MSL, while the average scheme achieves higher Rec and $F1_{PA}$ on MSL and SWAN.
3) *In Summary:* reconstruction error-based scoring is less effective in a two-layer All-MLP architecture, as shallow MLPs struggle to achieve accurate reconstruction.

3) Lightweight All-MLP Architecture: LFTSAD uses four parallel two-layer MLPs to construct a lightweight All-MLP architecture. This design ensures high timeliness and low resource consumption. To validate its effectiveness, two comparison schemes are designed by replacing the two-layer MLPs with two-layer RNNs and two-layer CNNs. Table. VI(c) lists the detection performance of three schemes.

1) *Overall Performance:* The two-layer MLP architecture consistently outperforms the other two. For instance, on MITDB, it achieves a Rec score of 1, compared with 0.8982 for the two-layer CNN and 0.5665 for the two-layer RNN.
2) *Stability*: The two-layer MLP architecture performs better across all the five datasets and three metrics.
3) *In Summary*: Within a shallow architecture, the MLP network demonstrates higher efficiency than both the RNNs and CNNs. This observation suggests that RNNs and CNNs require deeper structures and more parameters to fully exploit their capabilities.

H. RQ-5 (Sensitivity)

To address *RQ-5*, two key parameters are evaluated.

1) Parameter α: The parameter α, ranging from [0,1], balances point-level and sequence-level anomaly scores to generate the final score. When $\alpha = 1$, only the point-level score is used; when $\alpha = 0$, only the sequence-level score is applied. Fig. 9(a) shows plots of performance changes across five datasets under α parameters. In the figure, green lines represent univariate datasets, and red lines represent multivariate datasets.

1) *Focusing on Univariate Datasets:* All the green lines (univariate datasets) gradually increase at first, and then drop sharply. Their performance is relatively stable within [0, 0.4].
2) *Considering Multivariate Datasets:* All the red lines rise rapidly first and then remain stable. Their performance is relatively stable within [0.7, 1.0].
3) *In Summary:* The stable intervals for parameter α are [0, 0.4] for univariate datasets and [0.7, 1.0] for multivariate datasets. This finding suggests that point-level learning is more effective for univariate datasets, while sequence-level learning performs better on multivariate datasets. In univariate data, outliers typically deviate clearly from normal values, favoring point-level learning. In contrast, multivariate outliers often span multiple dimensions, making sequence-level learning more suitable.

2) Number of MLP Layers: It is another important parameter influencing timeliness and resource consumption of LFTSAD. Fewer layers result in a lighter model with fewer parameters, lower resource usage, and higher timeliness, but reduced representation capability. More layers improve representation but increase parameter count, resource consumption, and reduce timeliness. Fig. 9(b) displays how performance varies with different MLP layers. In the figure, performance improves initially, and then declines sharply as layers increase. The best results are achieved with two layers across all the datasets and metrics. Therefore, a two-layer MLP is optimal for the lightweight LFTSAD, as deeper networks increase overfitting risk and require higher dropout rates, while shallower ones limit learning capacity.

I. Discussion

This article focuses on high-speed, low-consumption, and high-accuracy timestamp-level anomaly detection on resource-limited edge devices. Extensive experiments demonstrate that the LFTSAD model successfully achieves all the three objectives.

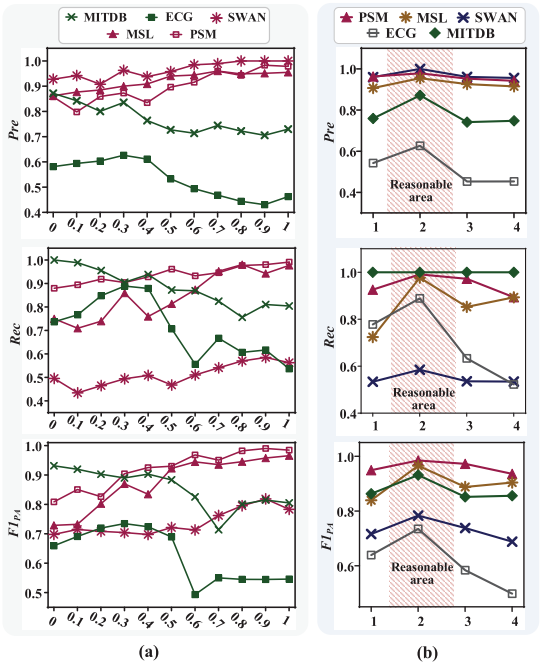

Fig. 9. Sensitivity analysis. (a) Parameter α. (b) Layer of MLP.

1) *High Speed:* Timeliness and deployability experiments show that LFTSAD requires significantly less detection time compared with deep TSAD models. Roughly speaking, LFTSAD is 3–10 times faster than other deep models. This advantage stems from the LFTSAD model being lightweight, with a shallow structure and few parameters, using only four parallel two-layer MLPs. In addition, the parallel learning process across M variables and two branches further enhances the speed of LFTSAD.
2) *Low Resource Consumption:* Attributed to its shallow and lightweight All-MLP architecture, LFTSAD exhibits significantly lower resource consumption compared with other deep models. Specifically, LFTSAD has only 126 K parameters, which is 1/10–1/50 of the parameter count of other deep models. On the Raspberry Pi 4b, the CPU usage of LFTSAD is only half that of other deep models. This efficiency allows LFTSAD to be deployed on resource-constrained devices like the Raspberry Pi 4b, which has only 2G of RAM.
3) *High Accuracy:* Accuracy and ablation experiments show that LFTSAD ranked first or second in over half of the eight metrics across most datasets. For example, LFTSAD ranked in the top two on seven metrics for the UCR and UCR-AUG datasets and six metrics for the MSL dataset. These results can be attributed to the accuracy-enhancing strategies: "point-level and sequence-level contrastive reconstruction learning" and "reconstruction discrepancy-based anomaly scoring."

VI. Conclusion

This article addresses the challenge of real-time, efficient anomaly detection in resource-constrained edge environments by the proposed LFTSAD—an unsupervised, lightweight, All-MLP-based anomaly detection model. Unlike traditional deep neural networks with large parameters, LFTSAD uses only four parallelizable two-layer MLP networks, offering a shallow architecture with few parameters. This design ensures high timeliness and low resource consumption, making it well-suited for deployment on resource-limited edge devices. In addition, LFTSAD integrates point-level and sequence-level perspectives to design a reconstruction discrepancy-based anomaly scoring scheme for accurate anomaly detection. Comparative evaluations on 14 real-world datasets and two edge platforms (Raspberry Pi 4b and Jetson Xavier NX) demonstrate its superiority. Specifically, LFTSAD operates 3–10 times faster than deep-learning-based SOTA models on Raspberry Pi 4b. Accuracy results further confirm its effectiveness, with LFTSAD ranking first or second in more than six metrics across the UCR, UCR_AUG, and MSL datasets.

However, the proposed LTFSAD method still has several limitations. First, the performance of LTFSAD shows sensitivity to the parameter α on certain datasets. Second, spatial correlations between variables are not explicitly considered in the current model. Third, LFTSAD currently lacks the capability for online updates and federated distributed deployment. To overcome these limitations, future work will focus on three directions. 1) developing a more robust and accurate All-MLP-based model by integrating time-frequency learning and applying pruning or quantization techniques; 2) designing a spatiotemporal joint learning framework based on the All-MLP architecture to capture both the temporal and spatial dependencies; and 3) building a real-time, online anomaly detection system that supports federated deployment on resource-constrained edge devices.

References

[1] G. Li and J. J. Jung, "Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges," *Inf. Fusion*, vol. 91, pp. 93–102, Mar. 2023.
[2] A. Garg, W. Zhang, J. Samaran, R. Savitha, and C.-S. Foo, "An evaluation of anomaly detection and diagnosis in multivariate time series," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2508–2517, Jun. 2022.
[3] G. Li, Z. Yu, K. Yang, M. Lin, and C. L. Philip Chen, "Exploring feature selection with limited labels: A comprehensive survey of semi-supervised and unsupervised approaches," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6124–6144, Nov. 2024.
[4] S.-E. Benkabou, K. Benabdeslem, V. Kraus, K. Bourhis, and B. Canitia, "Local anomaly detection for multivariate time series by temporal dependency based on Poisson model," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6701–6711, Nov. 2022.
[5] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–42, Oct. 2024.
[6] Z. Zhong, Z. Yu, Z. Fan, C. L. P. Chen, and K. Yang, "Adaptive memory broad learning system for unsupervised time series anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 26, 2024, doi: 10.1109/TNNLS.2024.3415621.
[7] W. Deng, J. Feng, and H. Zhao, "Autonomous path planning via sand cat swarm optimization with multi-strategy mechanism for unmanned aerial vehicles in dynamic environment," *IEEE Internet Things J.*, early access, Feb. 20, 2025, doi: 10.1109/JIOT.2025.3542587.
[8] D. Guo, Z. Zhang, B. Yang, J. Zhang, H. Yang, and Y. Lin, "Integrating spoken instructions into flight trajectory prediction to optimize automation in air traffic control," *Nature Commun.*, vol. 15, no. 1, p. 9662, Nov. 2024.
[9] T. Ergen and S. S. Kozat, "Unsupervised anomaly detection with LSTM neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 8, pp. 3127–3141, Aug. 2020.
[10] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, "Unsupervised deep anomaly detection for multi-sensor time-series signals," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
[11] F. Li, J. Chen, L. Zhou, and P. Kujala, "Investigation of ice wedge bearing capacity based on an anisotropic beam analogy," *Ocean Eng.*, vol. 302, Jun. 2024, Art. no. 117611.

[12] W. Li et al., "Fault diagnosis using variational autoencoder GAN and focal loss CNN under unbalanced data," *Struct. Health Monitor.*, vol. 24, no. 3, Jul. 2024, Art. no. 14759217241254121.
[13] M. Li, J. Li, Y. Chen, and B. Hu, "Stress severity detection in college students using emotional pulse signals and deep learning," *IEEE Trans. Affect. Comput.*, early access, Mar. 4, 2025, doi: 10.1109/TAFFC.2025.3547753.
[14] H. Chen, Y. Sun, X. Li, B. Zheng, and T. Chen, "Dual-scale complementary spatial–spectral joint model for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 6772–6789, 2025.
[15] N. Bai, X. Wang, R. Han, Q. Wang, and Z. Liu, "PAFormer: Anomaly detection of time series with parallel-attention transformer," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 3315–3328, Feb. 2025.
[16] R. Xu, H. Miao, S. Wang, P. S. Yu, and J. Wang, "PeFAD: A parameter-efficient federated framework for time series anomaly detection," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 3621–3632.
[17] M. Pietroń, D. Żurek, K. Faber, and R. Corizzo, "Towards efficient deep autoencoders for multivariate time series anomaly detection," in *Proc. Int. Conf. Comput. Sci.*, Jan. 2024, pp. 461–469.
[18] M. Pietroń, D. Żurek, K. Faber, A. Wójcik, and R. Corizzo, "AD-NEv+––The multi-architecture neuroevolution-based multivariate anomaly detection framework," in *Proc. Genetic Evol. Comput. Conf. Companion*, Jul. 2024, pp. 607–610.
[19] Z. Zhong, Z. Yu, Y. Yang, W. Wang, and K. Yang, "PatchAD: A lightweight patch-based MLP-mixer for time series anomaly detection," 2024, *arXiv:2401.09793*.
[20] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2020, pp. 3395–3404.
[21] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2021, pp. 1–20.
[22] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "DCdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. 29th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2023, pp. 3033–3045.
[23] Y. Nam et al., "Breaking the time-frequency granularity discrepancy in time-series anomaly detection," in *Proc. ACM Web Conf.*, May 2024, pp. 4204–4215.
[24] W. Xiong, P. Wang, X. Sun, and J. Wang, "SiET: Spatial information enhanced transformer for multivariate time series anomaly detection," *Knowl.-Based Syst.*, vol. 296, Jul. 2024, Art. no. 111928.
[25] J. Zhong et al., "A masked attention network with query sparsity measurement for time series anomaly detection," in *Proc. IEEE Int. Conf. Multimedia Expo (ICME)*, Jul. 2023, pp. 2741–2746.
[26] G. Sivapalan, K. K. Nundy, S. Dev, B. Cardiff, and D. John, "ANNet: A lightweight neural network for ECG anomaly detection in IoT edge sensors," *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 1, pp. 24–35, Feb. 2022.
[27] T. Zhou et al., "FiLM: Frequency improved legendre memory model for long-term time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 12677–12690.
[28] Z. Zhang, J. Wang, Y. Xia, D. Wei, and Y. Niu, "Solar-mixer: An efficient end-to-end model for long-sequence photovoltaic power generation time series forecasting," *IEEE Trans. Sustain. Energy*, vol. 14, no. 4, pp. 1979–1991, Apr. 2023.
[29] K. Yi et al., "Frequency-domain MLPs are more effective learners in time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 76656–76679.
[30] Y. Qin, H. Luo, F. Zhao, Y. Fang, X. Tao, and C. Wang, "Spatio-temporal hierarchical MLP network for traffic forecasting," *Inf. Sci.*, vol. 632, pp. 543–554, Jun. 2023.
[31] Z. Xu, A. Zeng, and Q. Xu, "FITS: Modeling time series with 10k parameters," in *Proc. 12th Int. Conf. Learn. Represent.*, Jan. 2023, pp. 1–24.
[32] L. Chen et al., "Frequency-domain spectrum discrepancy-based fast anomaly detection for IIoT sensor time-series signals," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–16, 2025.
[33] L. Chen, X. Cao, T. He, Y. Xu, X. Liu, and B. Hu, "A lightweight all-MLP time–frequency anomaly detection for IIoT time series," *Neural Netw.*, vol. 187, Jul. 2025, Art. no. 107400.
[34] L.-R. Yu, Q.-H. Lu, and Y. Xue, "DTAAD: Dual TCN-attention networks for anomaly detection in multivariate time series data," *Knowl.-Based Syst.*, vol. 295, Jul. 2024, Art. no. 111849.
[35] M. Pietroń, D. Żurek, K. Faber, and R. Corizzo, "AD-NEv: A scalable multilevel neuroevolution framework for multivariate anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Aug. 14, 2024, doi: 10.1109/TNNLS.2024.3439404.
[36] Y. Zheng et al., "Correlation-aware spatial–temporal graph learning for multivariate time-series anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11802–11816, Sep. 2024.
[37] H. Xu, Y. Wang, S. Jian, Q. Liao, Y. Wang, and G. Pang, "Calibrated one-class classification for unsupervised time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5723–5736, Nov. 2024.
[38] Q. Zhou, J. Chen, H. Liu, S. He, and W. Meng, "Detecting multivariate time series anomalies with zero known label," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 4, pp. 4963–4971.
[39] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, May 2021, pp. 4027–4035.
[40] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw. (ICANN)*, 2019, pp. 703–716.
[41] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
[42] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2018, pp. 387–395.
[43] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, Dec. 2008, pp. 413–422.
[44] J. Paparrizos, P. Boniol, T. Palpanas, R. S. Tsay, A. Elmore, and M. J. Franklin, "Volume under the surface: A new accuracy evaluation measure for time-series anomaly detection," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2774–2787, Jul. 2022.
[45] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, Jan. 2021, pp. 7194–7201.

Lei Chen (Member, IEEE) received the M.Sc. degree in computer science and engineering and the Ph.D. degree in automatic control and electrical engineering from Hunan University, Changsha, China, in 2012 and 2017, respectively.

He is currently an Associate Professor with the School of Information and Electrical Engineering, Hunan University of Science and Technology, Xiangtan, China. His current research interests include anomaly detection, lightweight design, time series analysis, deep learning, and big data analysis.

Jiajun Tang received the B.Eng. degree in automation from Hunan Institute of Science and Technology, Yueyang, China, in 2023. He is currently pursuing the master's degree in control science and engineering with Hunan University of Science and Technology, Xiangtan, China.

His current research interests include data-driven anomaly detection, deep learning, and fault diagnosis.

Ying Zou, photograph and biography not available at the time of publication.

Xuxin Liu, photograph and biography not available at the time of publication.

Xingquan Xie, photograph and biography not available at the time of publication.

Guangyang Deng, photograph and biography not available at the time of publication.